Fundación Universitaria Internacional de La Rioja (UNIR)

Facultad de Ingeniería

Ingeniería Informática

Planificación de una arquitectura agéntica verificable para facturación electrónica multipaís en Latinoamérica

Trabajo de Fin de Grado

Presentado por: De Mendoza Tovar, Alejandro

Director: Giraldo Hurtado, Johan Sebastián

Ciudad: Bogotá D. C.
Fecha: octubre de 2026

Resumen

La factura electrónica con validación previa es hoy la base de la fiscalización digital en Latinoamérica. Cada país define su propio formato, sus catálogos de códigos y sus reglas de validación, y los modifica por versiones. Para una organización multinacional que factura en varios países, esto significa que cada país o proveedor tecnológico nuevo obliga a construir de nuevo el mapeo entre su modelo de datos canónico y el formato nacional, una tarea manual de aproximadamente cien campos que produce defectos recurrentes y que solo se detectan cuando el proveedor o la autoridad tributaria rechazan el documento.

El propósito de este trabajo es planificar la implementación de una arquitectura agéntica verificable que reduzca el esfuerzo de incorporar nuevos países y proveedores sin comprometer la validez normativa de los comprobantes. La arquitectura combina dos planos: un plano de integración orientado a eventos y un plano agéntico en el que modelos de lenguaje de gran tamaño proponen mapeos, plantillas y pruebas. Entre ambos se ubica un kit de verificación determinista que decide qué propuesta puede llegar a revisión humana, porque valida el esquema vigente, los catálogos oficiales versionados y las reglas derivadas de una taxonomía propia de seis tipos de defecto de mapeo, y relaciona cada regla con la sección del documento normativo que la origina.

La metodología sigue la investigación en ciencia del diseño y un ciclo de vida por fases con entregables, y se apoya en el análisis de causas, la decisión multicriterio, las pruebas de mutación y la estimación de tres puntos con simulación de Monte Carlo. Un prototipo funcional del kit, evaluado con un banco de 22 defectos inyectados, detectó el 86 % de los defectos en ejecución y el 100 % con ejemplos validados en integración continua, sin falsos positivos, frente al 23 % de la validación de esquema. El plan de ejecución, con diez personas en ocho roles, estima un plazo de 40,2 semanas en el percentil 50 y de 42,2 en el percentil 80, con un costo de COP 1.013 millones y COP 1.059 millones, y un punto de equilibrio de 2,9 países para los agentes y de 5,4 para la inversión completa. Las conclusiones indican que la inteligencia artificial generativa puede aplicarse en un dominio regulado como la facturación electrónica si su salida se trata como una propuesta sujeta a verificación objetiva, trazabilidad hasta la norma y aprobación humana, y no como una decisión automática.

Palabras clave: facturación electrónica, agentes de inteligencia artificial, verificación determinista, arquitectura de software, planificación de proyectos.

Abstract

Clearance-based electronic invoicing is now the foundation of digital tax control in Latin America. Each country defines its own format, code lists and validation rules, and changes them through successive versions. For a multinational organization that issues invoices in several countries, every new country or e-invoicing provider means rebuilding the mapping between its canonical data model and the national format. This manual task covers about one hundred fields and produces recurring defects that are only detected when the provider or the tax authority rejects the document.

The purpose of this work is to plan the implementation of a verifiable agentic architecture that reduces the effort of onboarding new countries and providers without compromising the regulatory validity of the documents issued. The architecture combines two planes: an event-driven integration plane and an agentic plane in which large language models propose mappings, templates and tests. Between them sits a deterministic verification kit that decides which proposal may reach human review. It checks the current schema, the versioned official code lists and the rules derived from an original taxonomy of six mapping defect types, and it links every rule to the section of the regulatory document it comes from.

The methodology follows design science research and a phased life cycle with deliverables, supported by root cause analysis, multi-criteria decision analysis, mutation testing and three-point estimation with Monte Carlo simulation. A functional prototype of the kit, evaluated with a bank of 22 injected defects, detected 86% of the defects at run time and 100% with validated examples in continuous integration, with no false positives, against 23% for schema validation alone. The execution plan, with ten people in eight roles, estimates a duration of 40.2 weeks at the 50th percentile and 42.2 at the 80th, at a cost of COP 1,013 and 1,059 million, with a break-even point of 2.9 countries for the agents and 5.4 for the full investment. The conclusions indicate that generative artificial intelligence can be applied in a regulated domain such as electronic invoicing when its output is treated as a proposal subject to objective verification, traceability to the regulation and human approval, rather than as an automatic decision.

Keywords: electronic invoicing, artificial intelligence agents, deterministic verification, software architecture, project planning.

Índice de contenidos

1. Introducción 10
1.1. Planteamiento general: descripción y justificación del proyecto 11
1.1.1. Planteamiento del problema 11
1.1.2. Descripción y justificación 12
1.2. Objetivos del TFG 14
1.3. Elementos innovadores del proyecto 15
2. Descripción de la situación de partida y planteamiento del problema 17
2.1. Una aproximación a la empresa u organización 17
2.2. Identificación del problema 18
2.3. Análisis del problema 21
2.3.1. Contextualización 21
2.3.2. Antecedentes 22
2.3.3. Trabajos relacionados 23
2.3.4. Conclusiones del análisis 24
3. Metodología 25
3.1. Enfoque metodológico y fases del proyecto 25
3.2. Revisión de la literatura 26
3.3. Métodos de diagnóstico 26
3.3.1. Diagrama de Ishikawa 26
3.3.2. Análisis de Pareto 27
3.4. Método de decisión para seleccionar la solución 28
3.5. Métodos de diseño arquitectónico 28
3.6. Método de validación técnica 29
3.7. Métodos de planificación del proyecto 30
3.8. Trazabilidad entre objetivos, métodos y entregables 30
4. Desarrollo de la solución 32
4.1. Aportación: taxonomía de defectos y kit de verificación 32

4.2. Requisitos y atributos de calidad 33
4.2.1. Requisitos funcionales 33
4.2.2. Escenarios de atributos de calidad 34
4.2.3. Restricciones 34
4.3. Selección de la alternativa de solución 34
4.4. Arquitectura de referencia 35
4.4.1. Contexto del sistema (C4, nivel 1) 35
4.4.2. Contenedores (C4, nivel 2) 36
4.4.3. Stack tecnológico 37
4.4.4. Contratos e interfaces 38
4.4.5. Modelo de datos 38
4.4.6. Seguridad por diseño 38
4.5. Decisiones arquitectónicas 39
4.6. Perfiles de trabajo y matriz de responsabilidades 40
4.7. Planificación de tareas, esfuerzos y cronograma 41
4.7.1. Fases e hitos de control 41
4.7.2. Estructura de desglose del trabajo y estimaciones 41
4.7.3. Nivelación de recursos 42
4.7.4. Cronograma nivelado 43
4.8. Presupuesto 44
4.8.1. Referentes salariales y sensibilidad del presupuesto 45
4.9. Análisis de riesgos 46
4.10. Plan de calidad, integración y despliegue continuos y gobierno del proyecto 47
4.10.1. Plan de calidad y de pruebas 47
4.10.2. Integración y despliegue continuos con Jenkins 47
4.10.3. Gobierno del proyecto 48
4.11. Evaluación de la solución 48
5. Resultados obtenidos 50

5.1. Prueba de concepto: alcance, implementación y pruebas 50
5.1.1. Implementación 50
5.1.2. Pruebas automatizadas 51
5.2. Banco de defectos por inyección 51
5.3. Agente de mapeo y ciclo gobernado 53
5.4. Consolas de revisión 53
5.5. Pipeline, despliegue y madurez alcanzada 55
5.5.1. Madurez alcanzada 55
5.6. Resultados de la planificación 55
5.6.1. Análisis de incertidumbre 55
5.6.2. Punto de equilibrio 56
5.7. Amenazas a la validez 57
5.8. Cumplimiento de los objetivos 58
6. Conclusiones 59
6.1. Síntesis del trabajo 59
6.2. Respuesta a la pregunta guía 59
6.3. Discusión de los objetivos 60
6.4. Contribuciones del trabajo 60
6.5. Implicaciones para la práctica 61
6.6. Limitaciones 61
6.7. Líneas de trabajo futuro 62
Referencias Bibliográficas 64
Anexos 69
Anexo A. Decisiones arquitectónicas completas 69
Anexo B. Supuestos de planificación 74
Anexo C. Resultados detallados del banco de defectos 75

Índice de figuras

Figura 1. Arquitectura propuesta: flujos de integración, kit de verificación y plano agéntico 13

Figura 2. Arquitectura actual de integración: flujos asíncrono y síncrono 18

Figura 3. Flujos actuales de emisión asíncrona y síncrona, estados del comprobante y origen de los defectos 19

Figura 4. Fases de la metodología, objetivos y entregables 25

Figura 5. Diagrama Ishikawa rechazo de comprobantes por defectos de mapeo 27

Figura 6. Componentes del kit de verificación (C4, nivel 3) 33

Figura 7. Diagrama de contexto (C4, nivel 1) 35

Figura 8. Diagrama de contenedores (C4, nivel 2) 36

Figura 9. Secuencia del ciclo gobernado de una propuesta de IA 40

Figura 10. Carga semanal por rol tras la nivelación 43

Figura 11. Cronograma nivelado con ruta crítica, P50 y P80 44

Figura 12. Presupuesto por partida 45

Figura 13. Detección de defectos por tipo y mecanismo 52

Figura 14. Consola web de revisión con una propuesta bloqueada por el kit 54

Figura 15. Consola de escritorio JavaFX con una propuesta pendiente de revisión . 54

Figura 16. Distribución del plazo total: plan base y escenario con dos personas de QA 56

Figura 17. Índice de criticidad de los paquetes de trabajo 56

Figura 18. Punto de equilibrio en esfuerzo acumulado 57

Índice de tablas

Tabla 1. Análisis de Pareto de los defectos de mapeo del caso de referencia 12
Tabla 2. Puntos de dolor del proceso actual 20
Tabla 3. Taxonomía de defectos de mapeo 20
Tabla 4. Cobertura de las necesidades del problema por enfoque 24
Tabla 5. Protocolo de búsqueda de la literatura 26
Tabla 6. Métricas de evaluación derivadas con el enfoque GQM 29
Tabla 7. Trazabilidad entre objetivos, fases, métodos y entregables 30
Tabla 8. Requisitos funcionales 33
Tabla 9. Escenarios de atributos de calidad 34
Tabla 10. Matriz de decisión ponderada 34
Tabla 11. Microservicios del plano de integración 36
Tabla 12. Componentes del plano agéntico 37
Tabla 13. Lenguajes y su justificación 37
Tabla 14. Frameworks, librerías e infraestructura 37
Tabla 15. Tablas del núcleo del modelo de datos 38
Tabla 16. Resumen de las decisiones arquitectónicas 39
Tabla 17. Equipo del proyecto 40
Tabla 18. Matriz RACI 40
Tabla 19. Fases e hitos de control del proyecto 41
Tabla 20. Estructura de desglose del trabajo con estimaciones de tres puntos 42
Tabla 21. Presupuesto por partida (millones de COP) 44
Tabla 22. Consumo estimado de LLM por agente 45
Tabla 23. Salario base implícito por rol y referentes públicos del mercado colombiano 46
Tabla 24. Sensibilidad del costo del proyecto a las tarifas por rol 46
Tabla 25. Registro de riesgos 47
Tabla 26. Etapas del pipeline de Jenkins 48

Tabla 27. Plan de evaluación de la solución 48

Tabla 28. Componentes implementados en la prueba de concepto 50

Tabla 29. Suites de pruebas y resultados 51

Tabla 30. Mutantes detectados por tipo y mecanismo 51

Tabla 31. Resultados de la simulación de Monte Carlo 55

Tabla 32. Cumplimiento de los objetivos específicos y del objetivo general 58

Tabla 33. Costo empresa mensual por rol (supuesto) 74

Tabla 34. Otros supuestos del modelo 74

Tabla 35. Resultado por mutante 75

1. Introducción

Este capítulo presenta el problema que aborda el trabajo, la línea general de la solución que se planifica y los objetivos que guían su desarrollo. El punto de partida es una situación frecuente en las organizaciones que operan en varios países de Latinoamérica: la factura electrónica es obligatoria en casi toda la región, pero cada administración tributaria la regula con un formato, unos catálogos y unas reglas propias. La factura electrónica latinoamericana se considera uno de los aportes de la región a la fiscalidad internacional y el inicio de la digitalización de las administraciones tributarias (Barreix y Zambrano, 2018). Su éxito fiscal tiene, sin embargo, un costo técnico para las empresas, que deben integrar sus sistemas con cada modelo nacional.

En una organización multinacional, la emisión de comprobantes pasa por una plataforma de integración que recibe los documentos de los sistemas de origen en un modelo canónico propio, los transforma al formato que exige cada proveedor tecnológico y los envía para su validación ante la autoridad. La parte más costosa y frágil de esa cadena es el mapeo: la correspondencia campo a campo entre el modelo canónico y el formato del país, incluida la traducción de códigos internos a los códigos de los catálogos oficiales. Hoy ese mapeo se construye a mano para cada país y cada proveedor, se valida con pruebas manuales y depende del conocimiento de pocas personas.

Los modelos de lenguaje de gran tamaño ofrecen una oportunidad para acelerar esa tarea, porque pueden leer una especificación técnica y proponer correspondencias entre esquemas de datos (Parciak et al., 2024). También generan contenido plausible e incorrecto (Huang et al., 2025), algo inadmisible cuando el resultado es un documento tributario. Por eso la solución que se planifica no deja que la inteligencia artificial decida: los agentes proponen, un kit de verificación determinista comprueba cada propuesta contra el esquema, los catálogos y las reglas de la norma, y una persona aprueba antes de que el cambio llegue a producción.

El trabajo corresponde al tipo de planificación de un proyecto de innovación tecnológica en el ámbito profesional. Su alcance llega hasta el diseño de la arquitectura y la planificación de su implementación, y se complementa con un prototipo funcional que valida la viabilidad técnica de las decisiones críticas. El documento se organiza así: el capítulo 1 plantea el problema, los objetivos y los elementos innovadores; el capítulo 2 describe la situación de partida y analiza el problema a partir de sus antecedentes; el capítulo 3 presenta la metodología; el capítulo 4 desarrolla la solución; el capítulo 5 expone los resultados por objetivo; y el capítulo 6 recoge las conclusiones.

1.1. Planteamiento general: descripción y justificación del proyecto

1.1.1. Planteamiento del problema

La factura electrónica es obligatoria en la mayoría de los países de Latinoamérica. Según el Banco Interamericano de Desarrollo, más de quince países de la región la han implementado como instrumento para combatir el fraude y facilitar el cumplimiento tributario (Banco Interamericano de Desarrollo [BID], s. f.). El modelo regional es de control previo: el comprobante debe ser validado por la autoridad tributaria, o por un intermediario autorizado, antes de tener efectos fiscales (Barreix y Zambrano, 2018). Por esta razón, un error en el contenido de una factura no es un problema de calidad de datos, sino un documento rechazado o una contingencia fiscal.

La diversidad normativa es la primera fuente del problema. En Colombia, la Resolución 000165 de 2023 adoptó la versión 1.9 del anexo técnico de la factura electrónica de venta (Dirección de Impuestos y Aduanas Nacionales [DIAN], 2023), y la Resolución 000008 de 2024 modificó los plazos para su implementación (DIAN, 2024). En Perú, la Resolución de Superintendencia 188-2010/SUNAT marcó el inicio formal del sistema de emisión electrónica (Superintendencia Nacional de Aduanas y de Administración Tributaria [SUNAT], 2010; Cabrera et al., 2026). Cada país tiene su propio calendario, su esquema y sus catálogos, y no existe en la región un modelo semántico común como el que define la norma europea EN 16931 (Comité Europeo de Normalización [CEN], 2017).

La segunda fuente es el proceso con el que las empresas responden a esa diversidad. En el caso de referencia de este trabajo, una organización multinacional que emite del orden de cientos de miles de comprobantes al mes en varios países, cada incorporación de un país o de un proveedor tecnológico exige construir a mano el mapeo de aproximadamente cien campos entre el modelo canónico y el formato del proveedor, que puede ser JSON, texto plano por segmentos o XML basado en UBL 2.1 (OASIS, 2013). Estas cifras se presentan como órdenes de magnitud del caso de referencia, anonimizado por confidencialidad. La literatura regional confirma el patrón: las revisiones sobre la implementación de la factura electrónica en Latinoamérica señalan errores en la emisión y validación de comprobantes y la necesidad de soporte técnico permanente para que las empresas se adapten a la norma (Cabrera et al., 2026; Leguizamón y Lugo, 2024).

Para medir y clasificar el problema, los defectos de mapeo registrados en las incorporaciones del caso de referencia se agrupan en seis tipos, que constituyen una taxonomía propia del trabajo y se describen en el apartado 2.2. Como el volumen de defectos en un desarrollo de esta escala hace inviable un conteo exhaustivo, la tabla 1 los ordena según

su frecuencia relativa estimada por juicio experto, siguiendo el principio de Pareto: concentrar el esfuerzo en los pocos tipos que explican la mayor parte de los defectos.

Tabla 1. *Análisis de Pareto de los defectos de mapeo del caso de referencia*

Tipo	Defecto	Posición	Peso estimado	Peso acumulado
D1	Código de catálogo incorrecto	1	40,8 %	40,8 %
D4	Campo exigido ausente en el modelo canónico	2	24,2 %	65,0 %
D2	Campos intercambiados o importes incoherentes	3	15,8 %	80,8 %
D3	Literal mal formateado	4	10,3 %	91,1 %
D6	Carácter reservado en el contenido	5	6,1 %	97,2 %
D5	Campo obsoleto por versión del estándar	6	2,8 %	100,0 %

Fuente: estimación del autor por juicio experto, a partir de su experiencia en el proyecto de integración del caso de referencia; pesos calculados por el método de centroide de rangos (Barron y Barrett, 1996).

Los tres tipos más frecuentes, D1, D4 y D2, concentran el 80,8 % de los defectos estimados. Son los pocos vitales del análisis: códigos de catálogo equivocados, campos exigidos que el modelo canónico no trae y campos intercambiados. Por eso el kit de verificación prioriza sus reglas, y el banco de defectos les asigna más casos de prueba. Los pesos se derivan del orden de frecuencia mediante el método de centroide de rangos, que convierte una clasificación ordinal en pesos sin exigir mediciones exactas (Barron y Barrett, 1996).

Las áreas afectadas son tres. El área de integración, que asume semanas de trabajo por cada país o proveedor nuevo. El área tributaria, que debe interpretar la norma y validar catálogos sin una herramienta que conecte cada regla con su fuente. Y las áreas de negocio que emiten los documentos, que reciben rechazos tardíos y deben corregir los datos en origen. El problema que aborda este trabajo es, por tanto, cómo reducir el costo de incorporar un país o proveedor nuevo sin comprometer la validez normativa de lo emitido.

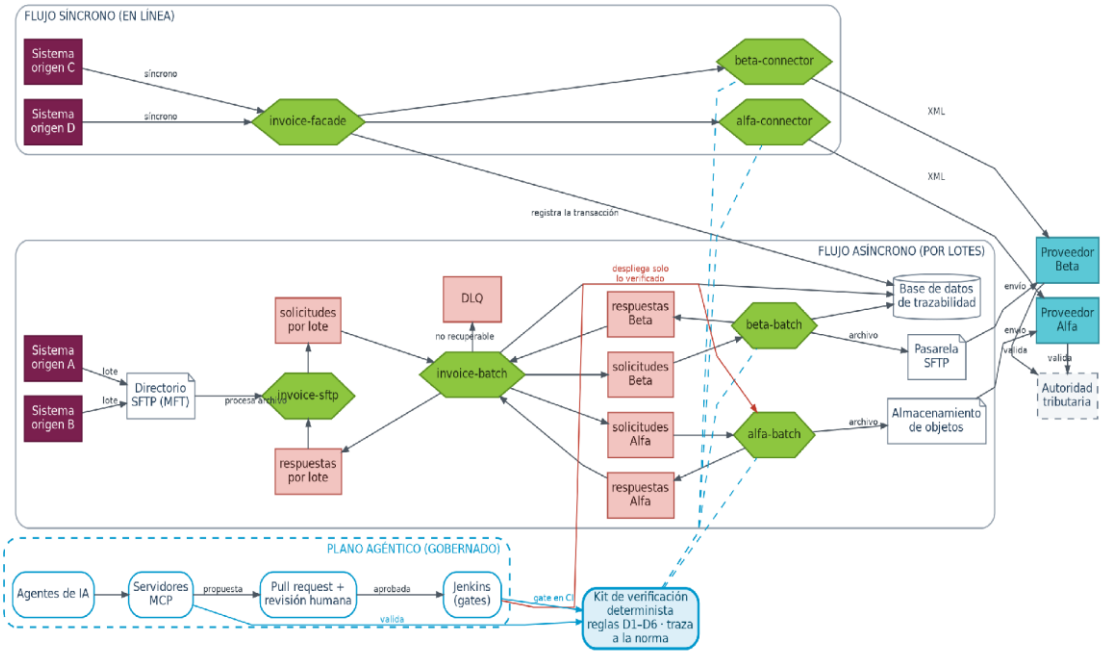
1.1.2. Descripción y justificación

La mejora que se propone es un proceso de incorporación de países y proveedores asistido por agentes de inteligencia artificial y gobernado por una verificación determinista. En lugar de que una persona escriba el mapeo desde cero, un agente lee la especificación técnica del proveedor, consulta los catálogos oficiales y propone la tabla de correspondencias, las plantillas y las pruebas. La propuesta no se acepta por sí misma: un kit de verificación la comprueba contra el esquema vigente, los catálogos versionados y las reglas de la taxonomía de defectos, y solo si la supera pasa a revisión humana. Lo aprobado se despliega por el mismo canal de integración y despliegue continuos que el resto del software.

La verificación se aplica en los dos canales de emisión de la plataforma. En el flujo asíncrono, el kit opera dentro de los adaptadores de cada proveedor antes de agrupar y enviar los lotes, de modo que ningún comprobante defectuoso llega a formar parte de un envío

masivo. En el flujo síncrono, opera dentro de los conectores en línea antes de invocar al proveedor, con un tiempo de respuesta acotado para no degradar la experiencia de quien espera el comprobante. Una misma regla protege así la emisión masiva y la unitaria, y los agentes proponen mapeos que se verifican contra ambos canales. La figura 1 muestra la arquitectura propuesta, con los dos flujos, el kit de verificación y el plano agéntico.

Figura 1. *Arquitectura propuesta: flujos de integración, kit de verificación y plano agéntico*



Fuente: elaboración propia.

La justificación económica se apoya en el carácter repetitivo del problema. El esfuerzo de incorporación se repite por cada país y por cada cambio de proveedor, de modo que crece de forma lineal con la expansión de la organización. Una inversión inicial en el kit de verificación y en los agentes solo se justifica si el ahorro por país compensa esa inversión a partir de un número razonable de incorporaciones; por eso el plan del proyecto incluye un modelo de punto de equilibrio cuyos resultados se presentan en el capítulo 5. Además, cada documento rechazado tiene un costo de corrección y un riesgo fiscal, y un control previo objetivo reduce ambos.

En el plano organizacional, la propuesta cambia el papel de los cargos involucrados. El desarrollador de integración deja de transcribir especificaciones y pasa a revisar propuestas con evidencia. El especialista tributario deja de validar documentos uno a uno y pasa a mantener reglas trazables a la norma. El líder técnico asume la aprobación de cada cambio propuesto por la inteligencia artificial, lo que exige definir responsabilidades explícitas mediante una matriz de asignación de responsabilidades.

En el plano de los procesos, el conocimiento que hoy es tácito, como las reglas escondidas en código heredado o los códigos acordados en reuniones, se convierte en reglas escritas, versionadas y verificables. En el plano de la información, cada regla queda ligada a la sección y a la huella digital del documento normativo del que proviene, de manera que un cambio de versión de la norma permite saber qué reglas revisar. Finalmente, la propuesta es coherente con la política pública colombiana: el documento CONPES 4144 de 2025 establece la Política Nacional de Inteligencia Artificial y señala la mitigación de riesgos y el uso ético y responsable de la inteligencia artificial como ejes estratégicos (Departamento Nacional de Planeación [DNP], 2025). Un proceso en el que la inteligencia artificial propone y las personas deciden, con verificación objetiva, es una aplicación concreta de ese principio.

1.2. Objetivos del TFG

Los objetivos se formulan con un solo verbo en infinitivo y de manera que los específicos sean los pasos necesarios para alcanzar el general. Cada objetivo específico tiene un resultado medible y un plazo dentro del calendario del Trabajo de Fin de Grado, y en el capítulo 5 se presentan los resultados obtenidos para cada uno.

Objetivo general

Planificar la implementación de una arquitectura agéntica verificable para la incorporación de nuevos países y proveedores de facturación electrónica en Latinoamérica, validando su viabilidad técnica mediante un prototipo funcional evaluado con un banco de defectos inyectados.

Objetivos específicos

OE1. Caracterizar los defectos de mapeo del proceso actual de integración de facturación electrónica mediante una taxonomía de al menos seis tipos, cada uno con su regla de verificación y su operador de mutación, durante las primeras tres semanas del proyecto.

OE2. Diseñar la arquitectura de referencia y el kit de verificación determinista, documentando al menos diez decisiones arquitectónicas y alcanzando en el prototipo una detección mínima del 80 % de los defectos inyectados en ejecución, sin falsos positivos, en un plazo de cuatro semanas.

OE3. Elaborar el plan de ejecución del proyecto, con estructura de desglose del trabajo, cronograma nivelado por recursos, presupuesto, registro de riesgos y estrategia de integración y despliegue continuos, cuantificando plazo y costo con un nivel de confianza del 80 % mediante simulación de Monte Carlo, al cierre del Trabajo de Fin de Grado.

El primer objetivo responde a la necesidad de medir el problema antes de resolverlo: sin una clasificación de los defectos no es posible saber qué debe verificar el kit ni cómo evaluarlo. El segundo objetivo produce el artefacto tecnológico central del trabajo, la arquitectura y su kit de verificación, y comprueba su viabilidad con un prototipo. El tercer objetivo convierte el diseño en un proyecto ejecutable, con equipo, calendario, costo y riesgos, que es la contribución propia de un trabajo de planificación.

De los objetivos se desprende la pregunta que orienta el trabajo: ¿en qué medida una arquitectura agéntica con verificación determinista reduce el esfuerzo de incorporar un nuevo país o proveedor de facturación electrónica, manteniendo o mejorando la tasa de comprobantes válidos frente al proceso manual?

El alcance se delimita así. El trabajo llega hasta el diseño de la arquitectura y la planificación de su implementación, y valida la viabilidad técnica de las decisiones críticas con un prototipo funcional construido sobre una especificación y datos sintéticos. No incluye la operación en producción ni la negociación con proveedores, y no usa datos tributarios reales, por razones de confidencialidad y de protección de datos.

1.3. Elementos innovadores del proyecto

El Manual de Oslo distingue entre la innovación de producto y la innovación de proceso, y exige que el cambio sea significativo frente a lo que la organización hacía antes (OCDE y Eurostat, 2006). Este proyecto es una innovación de proceso: cambia la forma en que la organización incorpora países y proveedores de facturación electrónica. Sus elementos innovadores son cinco, y cada uno representa una contribución concreta frente a lo que existe.

Primero, una taxonomía de defectos de mapeo con reglas y operadores de mutación. La literatura sobre factura electrónica en la región describe beneficios y desafíos de su implementación (Cabrera et al., 2026; Leguizamón y Lugo, 2024), pero no ofrece una clasificación operativa de los defectos de integración. La taxonomía D1 a D6 asocia a cada tipo de defecto una regla de verificación automática y un operador de mutación que permite inyectarlo de forma controlada. De este modo, la taxonomía sirve a la vez para diagnosticar el problema, para construir el verificador y para evaluarlo.

Segundo, la verificación determinista con procedencia normativa. Las propuestas que usan modelos de lenguaje para emparejar esquemas se evalúan con métricas de coincidencia y reconocen que sus resultados requieren verificación humana (Parciak et al., 2024; Narayan et al., 2022). Este proyecto agrega un verificador que no depende del juicio del modelo: cada regla del kit cita la sección y la huella digital del documento normativo del

que proviene. Un auditor puede rastrear cualquier rechazo hasta la norma, y un cambio de versión de la norma indica exactamente qué reglas revisar.

Tercero, un ciclo gobernado para la inteligencia artificial. Los agentes basados en modelos de lenguaje ya razonan y usan herramientas (Yao et al., 2023; Wang et al., 2024), y la revisión de Quevedo-Tumailli et al. (2025) concluye que, aunque mejoran la productividad en la generación de código, requieren una supervisión humana rigurosa. En la arquitectura propuesta, los agentes solo entregan propuestas en forma de solicitudes de cambio; no pueden desplegar ni escribir en producción, y una propuesta que no pasa el kit no puede aprobarse. Cada decisión queda registrada con su verificación, su revisor y su fecha.

Cuarto, una evaluación objetiva mediante un banco de defectos por inyección. Las pruebas de mutación miden la capacidad de un conjunto de pruebas para detectar defectos introducidos deliberadamente (Jia y Harman, 2011). El proyecto aplica esa técnica a los mapeos de facturación: a partir de un mapeo correcto se generan versiones defectuosas, una o más por tipo de la taxonomía, y se mide qué porcentaje detecta cada mecanismo de verificación. Así la contribución del kit se demuestra con datos y no con opiniones.

Quinto, una planificación cuantitativa bajo incertidumbre. En lugar de un cronograma con fechas fijas, el plan estima cada paquete de trabajo con tres valores, nivela la carga para que ningún rol supere su capacidad y simula el proyecto con el método de Monte Carlo, siguiendo los enfoques de gestión adaptativa de la guía del PMI (Project Management Institute [PMI], 2021). El resultado es un plazo y un costo acompañados de su probabilidad, un presupuesto que incluye el consumo de los modelos de lenguaje y un punto de equilibrio que indica a partir de cuántos países se recupera la inversión.

El alcance esperado en cada proceso es el siguiente. En la incorporación de países, reducir el esfuerzo manual de mapeo y medir esa reducción frente a una línea base. En la emisión, que el 100 % de los comprobantes pase la verificación antes de salir hacia el proveedor. En la gestión del conocimiento normativo, que cada regla sea trazable a su fuente. Y en la gestión del proyecto, que el plan comunique fechas y costos con su nivel de confianza.

2. Descripción de la situación de partida y planteamiento del problema

2.1. Una aproximación a la empresa u organización

La organización de referencia es una empresa multinacional de servicios con operación en varios países de Latinoamérica. Por razones de confidencialidad no se identifica la organización ni sus proveedores, y las cifras se expresan como órdenes de magnitud. Su facturación es masiva y recurrente: emite del orden de cientos de miles de comprobantes al mes, entre facturas, boletas y notas de crédito, con picos de volumen en los cierres de mes.

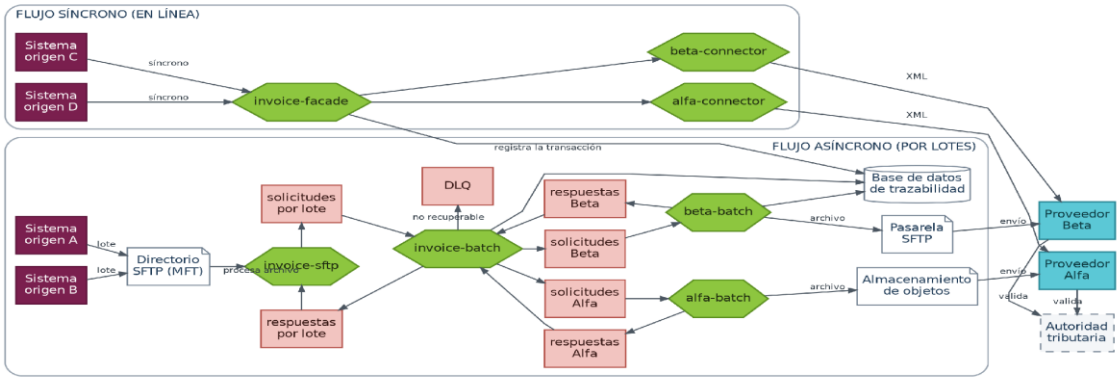
Desde la perspectiva de sistemas, la organización puede describirse en tres niveles. El primero son los sistemas de origen, que gestionan las ventas, los contratos y la contabilidad, y que producen la información de cada comprobante. El segundo es la plataforma de integración, un conjunto de microservicios que recibe los comprobantes, los enruta por país, los transforma al formato de cada proveedor y procesa las respuestas. El tercero son los sistemas externos: los proveedores tecnológicos de facturación autorizados en cada país y las administraciones tributarias que validan los documentos.

La plataforma de integración se organiza en subsistemas. El subsistema de ingesta recibe archivos por transferencia gestionada de archivos y solicitudes por servicios web, valida el modelo canónico y publica cada comprobante en un sistema de mensajería orientado a eventos. El subsistema de enrutamiento decide el proveedor según el país y el tipo de documento. Los adaptadores, uno por proveedor, transforman el comprobante al formato propietario, lo persisten, lo agrupan en lotes y lo envían. El subsistema de respuestas correlaciona el resultado del proveedor con el comprobante original y actualiza su estado, y el subsistema de archivo conserva los documentos durante el plazo legal. Esta forma de organización sigue los patrones clásicos de integración empresarial, como el modelo de datos canónico, el enrutador basado en contenido y el traductor de mensajes (Hohpe y Woolf, 2003).

La plataforma atiende dos modalidades de emisión que comparten la misma infraestructura. El flujo asíncrono, o por lotes, está pensado para la emisión masiva: los sistemas de origen depositan archivos con muchos comprobantes, que se procesan, se agrupan por proveedor y se envían sin que el sistema de origen espere la respuesta. El flujo síncrono, o en línea, está pensado para la emisión unitaria: un sistema de origen solicita un solo comprobante a la fachada de servicios web y espera, en la misma llamada, la respuesta del proveedor, como ocurre en una venta en mostrador o en la expedición inmediata de un documento. Los dos flujos se interconectan en tres puntos. Comparten la base de datos de

trazabilidad, en la que cada comprobante queda registrado con su identificador de correlación sin importar el canal por el que llegó. Comparten los proveedores tecnológicos, que reciben los comprobantes por lotes mediante archivos o de forma unitaria mediante sus conectores en línea. Y comparten las reglas de transformación de cada proveedor, de modo que un defecto de mapeo afecta por igual a la emisión masiva y a la unitaria. La figura 2 representa ambos flujos y sus puntos de interconexión.

Figura 2. *Arquitectura actual de integración: flujos asíncrono y síncrono*



Fuente: elaboración propia.

Los procesos donde se concentran los problemas son la incorporación de un país o proveedor y la corrección de rechazos. En la incorporación intervienen el equipo de integración, el especialista tributario, el proveedor tecnológico, los responsables de los sistemas de origen y los equipos de infraestructura y seguridad. El procedimiento escrito comprende la recepción de la especificación del proveedor, el diseño del mapeo, el desarrollo del adaptador, las pruebas en ambientes previos y la puesta en producción.

Junto al procedimiento escrito existe un procedimiento tácito que explica buena parte de los defectos. Incluye los códigos acordados en reuniones con el proveedor que no quedan en la especificación, las reglas implementadas años atrás en código heredado que nadie documentó y el conocimiento de las pocas personas que conocen cada formato. La gestión de credenciales, llaves y accesos de red también forma parte de este procedimiento: se tramita con varios actores y suele determinar el calendario más que el propio desarrollo. El análisis de la situación de partida debe considerar ambos procedimientos, porque la solución no solo automatiza el mapeo, sino que convierte en reglas explícitas el conocimiento que hoy es tácito.

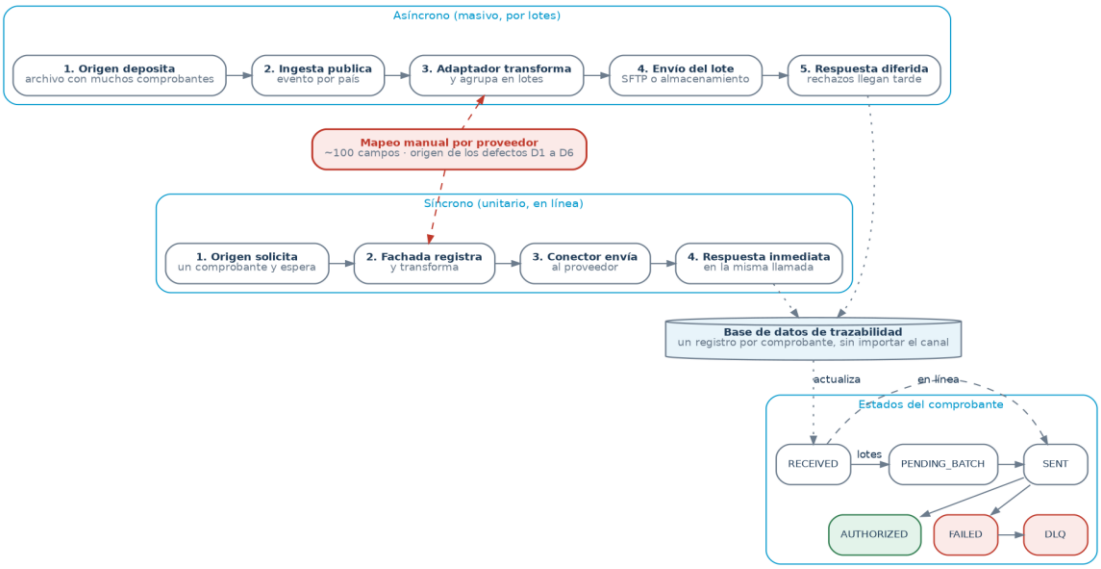
2.2. Identificación del problema

El proceso actual de emisión asíncrona sigue seis pasos: los sistemas de origen generan el comprobante en el modelo canónico; lo depositan mediante transferencia

gestionada de archivos; un servicio de ingesta lo publica en el sistema de mensajería, enrutado por país; un adaptador por proveedor lo transforma, lo persiste, lo agrupa en lotes y lo envía; el proveedor lo presenta ante la autoridad tributaria; y la respuesta regresa a la plataforma, donde el resultado del proveedor es la fuente primaria y la constancia oficial es el respaldo. Cada comprobante pasa por los estados recibido, pendiente de lote, enviado y autorizado o fallido, con reintentos controlados y una cola para los errores que no se pueden recuperar.

El proceso de emisión síncrona es más corto, pero más exigente en tiempo. El sistema de origen envía un comprobante a la fachada en línea; la fachada lo registra en la base de datos de trazabilidad, lo transforma con las reglas del proveedor y lo envía por el conector correspondiente; la respuesta regresa en la misma llamada y el comprobante pasa del estado recibido al de enviado y, de ahí, a autorizado o fallido, sin pasar por la agrupación en lotes. Por eso un mismo defecto de mapeo se manifiesta de forma distinta en cada canal. En el flujo asíncrono se descubre tarde, cuando el lote ya fue enviado y regresa con rechazos masivos que deben corregirse en origen. En el flujo síncrono se descubre de inmediato, pero el error llega al usuario en el momento mismo de la transacción. En ambos casos el origen es el mismo, el mapeo manual, y por eso la verificación que se planifica debe operar en los dos canales. La figura 3 resume los dos procesos, los estados del comprobante y el punto en que se originan los defectos.

Figura 3. *Flujos actuales de emisión asíncrona y síncrona, estados del comprobante y origen de los defectos*



Fuente: elaboración propia.

A partir de este flujo se identifican siete puntos de dolor, presentados en la tabla 2. Algunos son técnicos y otros de gestión, y no todos se resuelven con inteligencia artificial.

Tabla 2. *Puntos de dolor del proceso actual*

#	Punto de dolor	Efecto	Respuesta de la propuesta
P1	Mapeo manual de unos cien campos entre formatos	Semanas de trabajo por país y defectos recurrentes	Agente de mapeo y kit de verificación
P2	Formatos heterogéneos: JSON, texto por segmentos y XML	Un adaptador distinto por proveedor	Arquetipo de adaptador
P3	Datos de origen incompletos que se corrigen en origen	Rechazos que regresan tarde	Verificación en ejecución y clasificación de fallos
P4	Reglas ocultas en código heredado	Conocimiento tácito y riesgo de omisión	Agente de extracción de reglas con evidencia
P5	Credenciales, llaves, puertos y accesos	Del orden de una semana por gestión	Planificación temprana y en paralelo
P6	Coordinación entre muchos actores	Esperas y retrabajo	Matriz de responsabilidades y gestión de interesados
P7	El ciclo se repite por país y por proveedor	Costo que crece de forma lineal	Arquetipo, kit y agentes reutilizables

Fuente: elaboración propia.

El punto de dolor P1 es el núcleo técnico del problema, y sus defectos se clasifican en la taxonomía de la tabla 3. Cada tipo tiene una regla de verificación que el kit implementa y un operador de mutación que permite evaluarlo. Un detalle orienta el diseño: un código de catálogo válido pero equivocado no lo detecta la validación de esquema, porque en UBL los códigos son cadenas de texto libres (OASIS, 2013). Solo lo detecta una regla de negocio que conozca la semántica del catálogo, o la comparación con ejemplos validados por personas.

Tabla 3. *Taxonomía de defectos de mapeo*

Tipo	Defecto	Ejemplo genérico	Regla de verificación	Operador de mutación
D1	Código de catálogo incorrecto	Código tomado de una nota verbal y no de la especificación	Catálogo oficial versionado y coherencia entre campos	Sustituir por otro código, válido o inexistente
D2	Campos intercambiados	Monto, cantidad y precio unitario cruzados	Invariantes aritméticas y ejemplos validados	Permutar dos campos o tomar un total equivocado
D3	Literal mal formateado	Mayúsculas, espacios o nombre de parámetro	Literales y listas cerradas con comparación exacta	Alterar mayúsculas o añadir espacios
D4	Campo exigido ausente en el canónico	Dato obligatorio que el modelo canónico no trae	Cobertura del esquema destino y campos no vacíos	Eliminar el mapeo o mapear a un valor vacío
D5	Campo obsoleto por versión	Campo retirado en la versión vigente del estándar	Esquema vigente y lista de elementos retirados	Añadir un elemento de la versión anterior
D6	Carácter reservado en el contenido	Separador del formato dentro de un texto libre	Conteo de campos y datos adversarios	Desactivar el saneamiento de un texto libre

Fuente: elaboración propia.

El problema queda identificado en tres dimensiones. En la dimensión técnica, el mapeo manual introduce defectos que la validación de esquema no detecta. En la dimensión organizacional, el conocimiento necesario para evitarlos está repartido entre pocas personas

y documentos dispersos. Y en la dimensión de gestión, el calendario de cada incorporación depende de trámites de acceso que no son técnicos. La solución que se planifica ataca las dos primeras con la arquitectura y la tercera con el plan del proyecto.

2.3. Análisis del problema

Este apartado analiza los antecedentes del problema en cuatro bloques: la contextualización de los conceptos que usa el trabajo, los antecedentes en los ámbitos internacional, regional, nacional y local, los trabajos relacionados agrupados por tema y las conclusiones del análisis. La búsqueda de literatura se organiza con un protocolo sistemático basado en la declaración PRISMA 2020 (Page et al., 2021), cuyo protocolo se describe en el apartado 3.2.

2.3.1. Contextualización

Factura electrónica con validación previa. Es un comprobante fiscal digital que la autoridad tributaria, o un intermediario autorizado, valida antes de que tenga efectos legales. Inicialmente se concibió como un instrumento de control documental para evitar la omisión de ventas y la inclusión de compras falsas, y luego se extendió a otras áreas del control tributario (Barreix y Zambrano, 2018).

Estándar UBL y modelo semántico. UBL (Universal Business Language) es un estándar de OASIS que define documentos comerciales en XML, entre ellos la factura (OASIS, 2013). Varios países latinoamericanos lo usan como base de sus perfiles nacionales, a los que añaden extensiones, catálogos y reglas. En Europa, la norma EN 16931-1 define un modelo semántico común de la factura al que se asocian sintaxis concretas (CEN, 2017).

Integración empresarial orientada a eventos. Es la forma de conectar sistemas mediante mensajes asíncronos y patrones reutilizables, como el modelo de datos canónico, el enrutador basado en contenido y el traductor de mensajes (Hohpe y Woolf, 2003). En este trabajo, el traductor de mensajes es justamente el componente que hoy se escribe a mano.

Requisitos, calidad y verificación del software. La ingeniería de software distingue entre la validación, que comprueba que se construye el producto correcto, y la verificación, que comprueba que el producto cumple su especificación (Sommerville, 2011). Las pruebas y las revisiones técnicas son los mecanismos de aseguramiento de la calidad que hacen visible esa diferencia en la práctica (Pressman, 2010). El kit de verificación de este trabajo es un mecanismo de verificación automático: comprueba que cada comprobante cumple la especificación del país.

Modelos de lenguaje de gran tamaño y agentes. Un modelo de lenguaje de gran tamaño genera texto a partir de instrucciones en lenguaje natural. Un agente basado en

estos modelos alterna razonamiento y acciones sobre herramientas para cumplir un objetivo (Yao et al., 2023), y su diseño suele organizarse en perfil, memoria, planificación y acción (Wang et al., 2024). El Model Context Protocol estandariza la forma en que los agentes acceden a herramientas y datos externos (Model Context Protocol, 2026).

Alucinación. Es la tendencia de los modelos de lenguaje a producir contenido plausible pero falso o infiel a la fuente (Huang et al., 2025). La generación aumentada con recuperación reduce el problema al anclar la respuesta en documentos recuperados (Lewis et al., 2020), pero no lo elimina.

Integración y despliegue continuos. Son prácticas de la cultura DevOps que automatizan la construcción, las pruebas y el despliegue del software mediante un canal o pipeline. Giraldo-Hurtado (2023) caracterizó las herramientas que implementan la metodología GitOps y demostró, con un prototipo funcional, el proceso completo de integración y despliegue continuos de una aplicación. En este trabajo, ese canal es el punto donde el kit de verificación actúa como control obligatorio antes de cualquier despliegue.

2.3.2. Antecedentes

Ámbito internacional. La Organización para la Cooperación y el Desarrollo Económicos analiza la factura electrónica como una de las piezas de la administración tributaria digital (OCDE, 2022), y el Banco Interamericano de Desarrollo describe la evolución hacia una administración tributaria basada en datos, en la que la factura electrónica es un componente central (Calijuri, 2024). Europa optó por un modelo semántico común que facilita la interoperabilidad entre países (CEN, 2017). En paralelo, el campo de la inteligencia artificial ha producido marcos de gestión de riesgos, como el del Instituto Nacional de Estándares y Tecnología de los Estados Unidos, que organiza la gestión en las funciones de gobernar, mapear, medir y gestionar (National Institute of Standards and Technology [NIST], 2023).

Ámbito regional. Latinoamérica es pionera en factura electrónica. Barreix y Zambrano (2018) documentan la experiencia regional desde su implementación hasta su impacto recaudatorio. Las revisiones recientes recogen las experiencias de varios países (Leguizamón y Lugo, 2024) y evaluaciones de impacto como la de Ecuador, que analiza el efecto de la factura electrónica sobre el cumplimiento tributario (Ramírez-Álvarez et al., 2022). En Perú, la factura electrónica permitió automatizar controles y mejorar la confiabilidad de la gestión tributaria (De Velazco, 2016), aunque persisten retos técnicos y jurídicos para los contribuyentes (Cabrera et al., 2026). En Chile, la discusión sobre la equivalencia funcional entre el documento físico y el documento digital muestra que la

validez de un comprobante electrónico depende de que cumpla condiciones técnicas precisas (Escobar, 2019).

Ámbito nacional. En Colombia, la DIAN regula la factura electrónica mediante anexos técnicos versionados. La adopción de la versión 1.9 (DIAN, 2023) y la posterior modificación de sus plazos (DIAN, 2024) muestran que cada cambio normativo obliga a las empresas a ajustar sus sistemas. En materia de inteligencia artificial, el documento CONPES 4144 de 2025 fija la política nacional, con ejes de mitigación de riesgos y de uso y adopción responsable (DNP, 2025).

Ámbito local. En el caso de referencia, el antecedente directo es la incorporación sucesiva de países y proveedores con el procedimiento manual descrito en el apartado 2.1, y los defectos registrados en esas incorporaciones, resumidos en la tabla 1.

2.3.3. Trabajos relacionados

Los trabajos relacionados se agrupan en cuatro temas: a) emparejamiento de esquemas con modelos de lenguaje, b) modelos de lenguaje en la ingeniería de software, c) fiabilidad y verificación de sistemas basados en modelos de lenguaje, y d) tecnología aplicada a la factura electrónica.

a) Emparejamiento de esquemas con modelos de lenguaje. El emparejamiento de esquemas es un problema clásico de la integración de datos (Rahm y Bernstein, 2001). Narayan et al. (2022) mostraron que un modelo fundacional puede realizar tareas de preparación de datos, incluido el emparejamiento, sin entrenamiento específico. Parciak et al. (2024) evaluaron modelos de lenguaje para emparejar esquemas a partir de nombres y descripciones, y encontraron que aceleran el arranque del proceso, pero que el esfuerzo de verificación humana sigue siendo central. Liu et al. (2025) combinaron modelos pequeños y grandes para reducir el costo y mejorar la precisión. Estos trabajos producen correspondencias entre campos, no transformaciones ejecutables con reglas de negocio, y no se han aplicado a formatos tributarios con catálogos versionados por una autoridad.

b) Modelos de lenguaje en la ingeniería de software. La revisión sistemática de Hou et al. (2024) documenta el uso de modelos de lenguaje en todas las fases del ciclo de vida del software. En el ámbito hispanohablante, Quevedo-Tumailli et al. (2025) revisaron 27 estudios y concluyeron que estos modelos mejoran la calidad del código y la productividad, pero que su uso exige una supervisión humana rigurosa por el riesgo de generar errores o código inseguro. Jimenez et al. (2024) propusieron evaluar a los agentes de código con las pruebas del propio repositorio, es decir, con un oráculo ejecutable y no con el juicio del modelo.

c) Fiabilidad y verificación. La revisión de Huang et al. (2025) clasifica las causas de la alucinación y concluye que los sistemas con recuperación de documentos la reducen sin eliminarla. Las pruebas de mutación ofrecen una forma objetiva de medir la eficacia de un mecanismo de verificación (Jia y Harman, 2011). Ninguno de estos trabajos aborda la verificación de mapeos tributarios.

d) Tecnología aplicada a la factura electrónica. La literatura regional sobre factura electrónica se concentra en su impacto fiscal, su marco normativo y su adopción por las empresas (Barreix y Zambrano, 2018; Leguizamón y Lugo, 2024; Cabrera et al., 2026). En una búsqueda preliminar, los trabajos que combinan modelos de lenguaje y facturas se ocupan de extraer datos de documentos, no de integrar sistemas. La tabla 4 resume cómo responde cada enfoque a las necesidades del problema.

Tabla 4. *Cobertura de las necesidades del problema por enfoque*

Enfoque	Automatiza el mapeo	Garantiza validez normativa	Trazabilidad a la norma	Gobierno de la IA
Plataforma comercial de facturación	Parcial	Sí, en su formato	Opaca para el cliente	No aplica
Desarrollo tradicional	No	Sí, si se programan las reglas	Solo si se documenta	No aplica
Emparejamiento con modelos de lenguaje	Sí, correspondencias	No	No	No tratado
Agentes de código	Sí, código y pruebas	Solo con pruebas existentes	No	Parcial
Arquitectura propuesta	**Sí, gobernado**	**Sí, verificación obligatoria**	**Sí, regla a fuente**	**Sí, NIST y CONPES 4144**

Fuente: elaboración propia a partir de los trabajos revisados.

2.3.4. Conclusiones del análisis

El análisis permite extraer tres conclusiones. La primera es que el problema es estructural en la región: la factura electrónica con validación previa es obligatoria, sus formatos son nacionales y cambian por versiones, y no existe un modelo semántico común que evite repetir la integración en cada país. La segunda es que los modelos de lenguaje pueden acelerar el mapeo, pero la literatura coincide en que sus resultados requieren verificación y supervisión humana, y en que la alucinación no desaparece con las técnicas actuales. La tercera es que existe un vacío: no se encontraron propuestas que combinen la generación de mapeos con inteligencia artificial, una verificación determinista trazable a la norma y un plan de proyecto que gobierne ambos. Ese vacío es el espacio que ocupa este trabajo.

3. Metodología

Este capítulo describe cómo se desarrolla el trabajo y justifica cada método seleccionado. Los métodos no se eligen por costumbre, sino porque responden a una necesidad concreta de los objetivos específicos: diagnosticar el problema con evidencia, decidir la solución sin subjetividad, diseñarla de forma verificable y planificar su ejecución con plazos y costos cuantificados. Al final del capítulo, la tabla 7 resume la trazabilidad entre objetivos, fases, métodos y entregables.

3.1. Enfoque metodológico y fases del proyecto

El trabajo sigue el enfoque de investigación en ciencia del diseño (design science research), propio de los sistemas de información, que estudia la creación de artefactos que resuelven problemas relevantes y exige evaluarlos con rigor (Hevner et al., 2004). Este enfoque es adecuado porque la contribución del trabajo es un artefacto tecnológico, una arquitectura con su kit de verificación y su plan de implementación, y no la explicación de un fenómeno. Se adopta el proceso de seis actividades de Peffers et al. (2007): identificar el problema, definir los objetivos de la solución, diseñar y desarrollar, demostrar, evaluar y comunicar.

Para organizar el trabajo en el tiempo, el proceso se concreta en un ciclo de vida por fases con un entregable al final de cada una, similar al modelo híbrido, secuencial por fases e iterativo dentro de ellas, que Giraldo-Hurtado (2023) aplicó para construir y validar un catálogo de herramientas con un prototipo funcional. Cada fase responde a un objetivo específico, de modo que el avance del trabajo se puede verificar objetivo por objetivo. El enfoque combina técnicas cualitativas, como el análisis de causas y el juicio experto, con técnicas cuantitativas, como la simulación y la medición de la detección de defectos, lo que corresponde a un diseño mixto (Hernández Sampieri et al., 2014). La figura 4 presenta las fases.

Figura 4. *Fases de la metodología, objetivos y entregables*



Fuente: elaboración propia.

La fase 0 delimita el problema y construye el estado de la cuestión. La fase 1 diagnostica las causas de los defectos y produce la taxonomía. La fase 2 selecciona la

alternativa de solución, diseña la arquitectura y la valida con un prototipo. La fase 3 elabora el plan de ejecución del proyecto. La fase 4 evalúa los resultados frente a las métricas definidas y redacta las conclusiones. Dentro de las fases 2 y 3 el trabajo es iterativo: el diseño y el plan se ajustan a partir de los resultados del prototipo y de la simulación.

3.2. Revisión de la literatura

La revisión de la literatura sigue un protocolo de búsqueda inspirado en la declaración PRISMA 2020, que recomienda documentar las fuentes, las cadenas de búsqueda y los criterios de selección para que la revisión sea transparente y reproducible (Page et al., 2021). No se trata de una revisión sistemática completa, que excede el alcance de un trabajo de planificación, sino de una búsqueda estructurada que garantiza que los antecedentes del capítulo 2 provienen de fuentes verificables. La tabla 5 resume las fuentes y cadenas utilizadas.

Tabla 5. *Protocolo de búsqueda de la literatura*

Fuente	Cadena o criterio de búsqueda	Propósito
IEEE Xplore, ACM Digital Library, Scopus	("schema matching" OR "data mapping") AND ("large language model" OR LLM)	Emparejamiento de esquemas con modelos de lenguaje
IEEE Xplore, ACM Digital Library, ScienceDirect	("LLM-based agent" OR "AI agent") AND ("software engineering" OR "code generation")	Agentes y modelos de lenguaje en ingeniería de software
Scopus, ScienceDirect	(hallucination OR "factual consistency") AND ("large language model") AND (verification OR survey)	Fiabilidad y verificación
Redalyc, SciELO, Dialnet, Google Académico	("factura electrónica" OR "facturación electrónica") AND ("América Latina" OR Latinoamérica)	Factura electrónica en la región
Sitios oficiales (DIAN, SUNAT, BID, CIAT, OCDE)	Normativa, anexos técnicos e informes vigentes	Marco normativo y antecedentes
Re-UNIR y repositorios institucionales	("DevOps" OR "GitOps" OR "integración continua") en trabajos de grado y maestría	Antecedentes académicos de integración y despliegue

Fuente: elaboración propia.

Se incluyen publicaciones revisadas por pares, normativa oficial y documentos técnicos de organismos reconocidos, en español o en inglés, publicados desde 2018, salvo los trabajos fundacionales de cada tema. Se excluyen los textos de opinión, los duplicados, los trabajos sin acceso al texto completo y los que tratan los modelos de lenguaje solo para extraer datos de facturas escaneadas, porque no abordan la integración entre sistemas. Los trabajos seleccionados se agrupan en los cuatro temas del apartado 2.3.3.

3.3. Métodos de diagnóstico

3.3.1. Diagrama de Ishikawa

El diagrama de causa y efecto, propuesto por Kaoru Ishikawa, organiza las causas posibles de un problema en categorías que convergen en un efecto, y es una de las

herramientas básicas del control de calidad (Ishikawa, 1986). Se selecciona porque el rechazo de comprobantes no tiene una causa única: intervienen el método de trabajo, las personas, la normativa, los datos, los sistemas y la forma de medir. El diagrama obliga a considerar todas esas dimensiones antes de proponer una solución y evita atribuir el problema solo a la tecnología.

Las categorías clásicas del método se adaptan al dominio de la integración de facturación electrónica: método, personas, normativa, datos, sistemas y medición. Las causas de cada categoría se identifican a partir de la descripción del proceso actual (apartados 2.1 y 2.2) y de la experiencia del autor en el caso de referencia. La figura 5 presenta el diagrama resultante.

Figura 5. *Diagrama Ishikawa rechazo de comprobantes por defectos de mapeo*



Fuente: elaboración propia.

El diagrama muestra que las causas se refuerzan entre sí. El mapeo manual (método) depende de conocimiento tácito (personas) para interpretar anexos técnicos versionados (normativa), y la única barrera antes del envío es la validación de esquema (medición), que no detecta códigos válidos pero equivocados. Esta lectura justifica que la solución combine un cambio de método, la propuesta asistida por agentes, con un cambio de medición, la verificación determinista, y no solo una de las dos.

3.3.2. Análisis de Pareto

El análisis de Pareto ordena las causas o los tipos de defecto según su frecuencia para concentrar el esfuerzo en los pocos que explican la mayor parte del problema, principio que Juran (1990) denominó la separación de los pocos vitales de los muchos útiles. Se selecciona

porque el trabajo necesita priorizar: no todas las reglas del kit de verificación tienen el mismo valor, y el banco de defectos debe asignar más casos a los tipos más frecuentes.

En un desarrollo de la escala del caso de referencia, el volumen diario de incidencias hace inviable un conteo exhaustivo de defectos por tipo. Por eso el análisis se basa en juicio experto: el autor ordena los seis tipos de la taxonomía según su frecuencia observada, y el orden se convierte en pesos mediante el método de centroide de rangos, que asigna a cada posición un peso derivado de su lugar en la clasificación y ha mostrado buen desempeño frente a otros métodos de ponderación por rangos (Barron y Barrett, 1996). El resultado se presenta en la tabla 1 y se declara como estimación, no como medición.

3.4. Método de decisión para seleccionar la solución

La selección de la alternativa de solución se realiza con un análisis de decisión multicriterio, en la forma de una matriz de decisión ponderada. El método consiste en definir criterios, asignarles pesos, puntuar cada alternativa en cada criterio y calcular un total ponderado (Romero, 1993). Se selecciona porque elimina la subjetividad de una elección por preferencia: los criterios y los pesos quedan escritos y pueden discutirse, y el resultado se puede reproducir. Es, además, el procedimiento que propone el área de análisis y resolución de decisiones del modelo CMMI para elegir entre alternativas técnicas mediante criterios establecidos (CMMI Product Team, 2010).

Se comparan cinco alternativas: una plataforma comercial de facturación, una plataforma de integración como servicio, el desarrollo tradicional sin inteligencia artificial, el uso de modelos de lenguaje sin verificación y la arquitectura propuesta. Los criterios son la validez normativa de lo emitido, el esfuerzo por país o proveedor nuevo, el control y la trazabilidad, el costo total de propiedad, el riesgo de implantación y la independencia del fabricante. Para comprobar que la decisión es robusta, se aplica un análisis de sensibilidad que varía el peso del criterio más débil de la alternativa ganadora y determina a partir de qué valor cambiaría la decisión.

3.5. Métodos de diseño arquitectónico

El diseño de la arquitectura combina tres métodos complementarios. El primero es el levantamiento de requisitos funcionales a partir del análisis del proceso actual, siguiendo las prácticas del análisis y diseño de sistemas (Kendall y Kendall, 2011). El segundo es la especificación de los atributos de calidad como escenarios medibles, con la forma estímulo, respuesta y medida (Bass et al., 2021), clasificados según el modelo de calidad de producto de ISO/IEC 25010 (ISO/IEC, 2023). Este método se selecciona porque convierte requisitos

difusos, como que el sistema sea confiable, en metas verificables, como cero comprobantes perdidos ante un reproceso.

El tercero es la documentación de la arquitectura con el modelo C4, que la describe en niveles de contexto, contenedores y componentes para audiencias distintas (Brown, s. f.), y el registro de cada decisión relevante como un registro de decisión arquitectónica, que documenta el contexto, las opciones, la decisión y sus consecuencias (Nygard, 2011). Estos métodos se seleccionan porque permiten justificar cada elemento de la arquitectura con un escenario de calidad y una decisión documentada, y no solo describirlo.

3.6. Método de validación técnica

La viabilidad técnica del diseño se valida con un prototipo funcional del kit de verificación y del ciclo gobernado de propuestas, construido sobre una especificación y datos sintéticos para no usar información de ninguna organización. El prototipo no sustituye al proyecto planificado: su función es reducir el riesgo técnico de las decisiones críticas antes de comprometer recursos.

Para evaluar el prototipo se aplica la técnica de pruebas de mutación, que mide la capacidad de un mecanismo de verificación para detectar defectos introducidos de forma deliberada (Jia y Harman, 2011). A partir de un mapeo correcto se generan versiones defectuosas, una o más por cada tipo de la taxonomía, y se mide qué porcentaje detecta cada mecanismo: la validación de esquema, el kit en ejecución y el kit con ejemplos validados en el canal de integración continua. Se selecciona porque produce una medida objetiva y reproducible de la eficacia del kit, en lugar de una apreciación.

Las métricas de la evaluación se derivan con el enfoque objetivo, pregunta y métrica (GQM), que exige que cada métrica responda a una pregunta asociada a un objetivo, para no medir por medir (Basili et al., 1994). La tabla 6 presenta las métricas del trabajo.

Tabla 6. *Métricas de evaluación derivadas con el enfoque GQM*

Objetivo	Pregunta	Métrica	Meta
OE1 · Caracterizar	¿Qué tipos de defecto concentran la mayoría de los rechazos?	Peso relativo por tipo de defecto (análisis de Pareto)	Identificar los tipos que suman al menos el 80 %
OE2 · Diseñar	¿Qué proporción de los defectos detecta el kit de verificación?	Tasa de detección por tipo: defectos detectados / defectos inyectados	Al menos 80 % en ejecución y 100 % con fixtures de integración continua
OE2 · Diseñar	¿Rechaza el kit comprobantes correctos?	Falsos positivos sobre salidas correctas	Cero
OE3 · Elaborar	¿Cuál es el plazo y el costo del proyecto con un nivel de confianza conocido?	Duración y costo en el percentil 80 de la simulación	Plazo P80 menor o igual a 12 meses

Objetivo	Pregunta	Métrica	Meta
Objetivo general	¿Se recupera la inversión frente al proceso manual?	Punto de equilibrio n* en número de países o proveedores	Calcular n* y contrastarlo con los países previstos

Fuente: elaboración propia.

3.7. Métodos de planificación del proyecto

La planificación del proyecto, contribución central de un trabajo de este tipo, se construye con cinco métodos encadenados. Primero, la estructura de desglose del trabajo descompone el proyecto en paquetes de trabajo con un entregable verificable cada uno (PMI, 2021). Segundo, cada paquete se estima con la técnica PERT, que usa tres valores, optimista, más probable y pesimista, para reflejar la incertidumbre de cada estimación (Malcolm et al., 1959). La duración esperada de cada paquete es $t_e = (o + 4m + p) / 6$.

Tercero, el cronograma se nivela por recursos con un esquema serial de generación de cronogramas, que programa los paquetes en orden de prioridad y los retrasa cuando un rol supera su capacidad (Kolisch, 1996). Este paso se incluye porque un plan que asigna a una persona más trabajo del que puede hacer no es ejecutable. Cuarto, sobre la red nivelada se ejecuta una simulación de Monte Carlo que repite el cálculo del proyecto miles de veces con duraciones aleatorias, y entrega el plazo y el costo como distribuciones de probabilidad (Gray y Larson, 2009). Esto permite comunicar fechas y presupuestos con su nivel de confianza, por ejemplo el percentil 80, y calcular la reserva de contingencia.

Quinto, los riesgos se gestionan con un registro que incluye los riesgos propios de la inteligencia artificial, organizados según las funciones de gobernar, mapear, medir y gestionar del marco del NIST (2023) y alineados con los ejes de mitigación de riesgos de la política nacional de inteligencia artificial (DNP, 2025). Se selecciona este marco porque trata el riesgo de la inteligencia artificial como parte de la gestión del proyecto, con responsables y respuestas, y no como una advertencia técnica.

3.8. Trazabilidad entre objetivos, métodos y entregables

La tabla 7 resume cómo se relacionan los objetivos con las fases, los métodos y los entregables. Cada objetivo específico tiene al menos un método que lo desarrolla y un entregable que lo evidencia, lo que permite verificar en las conclusiones el cumplimiento de cada uno.

Tabla 7. *Trazabilidad entre objetivos, fases, métodos y entregables*

Objetivo	Fase	Métodos	Entregable	Apartado
OE1 · Caracterizar los defectos de mapeo	1. Diagnóstico	Ishikawa, Pareto por juicio experto y centroide de rangos, taxonomía	Causas priorizadas y taxonomía D1 a D6	2.2 y 4

Objetivo	Fase	Métodos	Entregable	Apartado
OE2 · Diseñar la arquitectura y el kit	2. Diseño y validación	Decisión multicriterio, C4, ADR, escenarios de calidad, prototipo, banco de defectos	Arquitectura de referencia, kit y resultados del banco	4 y 5
OE3 · Elaborar el plan de ejecución	3. Planificación	EDT, PERT, nivelación de recursos, Monte Carlo, gestión de riesgos de IA	Plan con plazo y costo P80, presupuesto y riesgos	4 y 5
Objetivo general	4. Evaluación y cierre	GQM y matriz de cumplimiento de objetivos	Resultados y conclusiones	5 y 6

Fuente: elaboración propia.

4. Desarrollo de la solución

Este capítulo desarrolla la solución que se planifica. Parte de la aportación central, el kit de verificación (apartado 4.1), y la sitúa en su contexto: los requisitos y atributos de calidad que debe cumplir (4.2), la selección de la alternativa (4.3) y la arquitectura de referencia con sus decisiones (4.4 y 4.5). Después desarrolla lo que constituye la contribución propia de un trabajo de planificación: los perfiles del equipo (4.6), la planificación de tareas, esfuerzos y cronograma (4.7), el presupuesto (4.8), el análisis de riesgos (4.9), el plan de calidad, la integración y despliegue continuos y el gobierno del proyecto (4.10) y la evaluación de la solución (4.11).

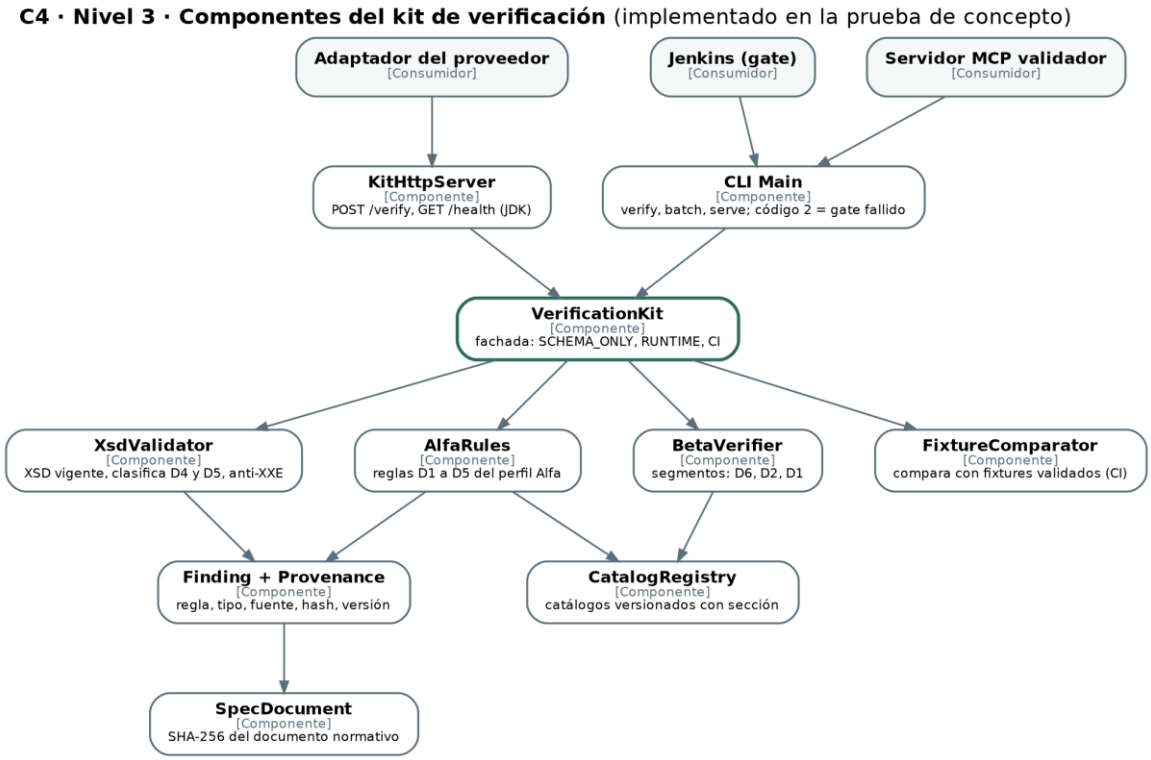
4.1. Aportación: taxonomía de defectos y kit de verificación

La aportación central del trabajo es un **kit de verificación determinista** construido a partir de la taxonomía de defectos de mapeo D1 a D6 caracterizada en el capítulo 2. El kit recibe un comprobante transformado al formato del proveedor y lo somete a tres controles sucesivos: la validación contra el esquema vigente, las reglas de negocio derivadas de cada tipo de defecto y, en el canal de integración continua, la comparación con ejemplos validados por personas. Cada hallazgo informa la regla que lo produjo, el tipo de defecto, la sección del documento normativo de la que proviene y la huella digital de ese documento, de modo que cualquier rechazo puede rastrearse hasta la norma.

El kit opera en dos modos. En **modo de ejecución**, se integra en los adaptadores del flujo asíncrono y en los conectores del flujo síncrono, y bloquea cualquier comprobante defectuoso antes de que salga hacia el proveedor. En **modo de integración continua**, actúa como control obligatorio sobre las propuestas de los agentes: una propuesta de mapeo que no supera el kit no puede aprobarse ni desplegarse. Para validar su viabilidad técnica se construyó un prototipo funcional del kit en Java 21, sin dependencias externas, junto con un agente de mapeo que propone correspondencias a partir de la especificación. El prototipo trabaja sobre una especificación y datos sintéticos, para no usar información de ninguna organización.

El kit es el componente más crítico, y por eso es el que implementa la prueba de concepto. La figura 6 muestra su estructura interna. Una fachada ofrece tres modos: solo esquema, ejecución y CI. Cada hallazgo lleva la regla que lo produjo, su tipo en la taxonomía, la sección de la especificación, el hash SHA-256 del documento normativo y la versión de la regla.

Figura 6. *Componentes del kit de verificación (C4, nivel 3)*



Fuente: elaboración propia. Elaboración propia. Componentes implementados en la prueba de concepto.

4.2. Requisitos y atributos de calidad

4.2.1. Requisitos funcionales

Tabla 8. *Requisitos funcionales*

ID	Requisito	Prioridad
RF-01	Recibir comprobantes por SFTP y REST, validar el canónico con JSON Schema y publicarlos por país	Alta
RF-02	Enrutar cada comprobante al proveedor según país, tipo de documento y una tabla versionada	Alta
RF-03	Transformar al formato del proveedor y verificar con el kit antes de enviar	Alta
RF-04	Procesar respuestas con fuente primaria y respaldo, y actualizar el estado	Alta
RF-05	Archivar PDF, XML y constancia con inmutabilidad y retención por país	Alta
RF-06	Consultar estado y auditoría de cada comprobante por su identificador de correlación	Media
RF-07	Generar desde la especificación del proveedor una propuesta de mapeo, plantillas y pruebas como pull request	Alta
RF-08	Extraer reglas de código heredado con evidencia de archivo y línea	Media
RF-09	Clasificar los fallos de la DLQ según D1 a D6 y proponer la corrección en origen	Media
RF-10	Registrar cada propuesta de IA con su verificación, revisor y decisión; impedir aprobar lo que no pasó el kit	Alta

Fuente: elaboración propia.

4.2.2. Escenarios de atributos de calidad

Cada escenario sigue la forma estímulo, respuesta y medida (Bass et al., 2021) y se clasifica según ISO/IEC 25010 (ISO/IEC, 2023).

Tabla 9. *Escenarios de atributos de calidad*

Atributo	Estímulo	Medida de respuesta
Eficiencia de desempeño	Día pico de emisión	Al menos 200.000 comprobantes procesados sin degradar el flujo síncrono
Eficiencia de desempeño	Solicitud síncrona	p95 menor a 3 s, sin contar el tiempo del proveedor
Fiabilidad	Operación mensual	99,9 % de disponibilidad en fachada e ingesta
Fiabilidad	Reproceso o mensaje duplicado	Cero pérdidas y cero duplicados gracias a la idempotencia
Adecuación funcional	Comprobante listo para salir	100 % pasa XSD y reglas antes del envío
Mantenibilidad	Nuevo proveedor	Incorporación con arquetipo y agente en una fracción del esfuerzo manual, medida con la línea base
Seguridad	Archivo comprimido malicioso, XXE o secreto expuesto	Rechazo por límites y por configuración segura; cero secretos en código o imágenes; agentes con mínimo privilegio
Seguridad (responsabilidad)	Auditoría de una regla o propuesta de IA	Trazable hasta su fuente normativa, su versión y su aprobador

Fuente: elaboración propia.

4.2.3. Restricciones

- Los agentes no pueden desplegar ni escribir en producción.
- Los datos tributarios reales no salen del perímetro de la organización; el desarrollo y la evaluación usan datos sintéticos.
- La retención del archivo legal cumple el plazo que fije cada país.
- Las credenciales se gestionan en un almacén de secretos con identidad de carga de trabajo.

4.3. Selección de la alternativa de solución

Cada alternativa se puntúa de 1 (peor) a 5 (mejor) en seis criterios ponderados. La arquitectura propuesta obtiene 4,05 sobre 5, frente a 3,35 del desarrollo tradicional, la segunda opción. Las puntuaciones son una valoración del autor y se validarán con un panel de expertos en un trabajo posterior.

Tabla 10. *Matriz de decisión ponderada*

Criterio	Peso	Plataforma comercial	iPaaS	Desarrollo tradicional	LLM sin verificación	Propuesta
Validez normativa de lo emitido	25 %	4	3	4	1	5

Criterio	Peso	Plataforma comercial	iPaaS	Desarrollo tradicional	LLM sin verificación	Propuesta
Esfuerzo por país o proveedor nuevo	20 %	4	2	1	4	4
Control y trazabilidad	15 %	2	3	4	1	5
Costo total de propiedad	15 %	2	3	3	4	3
Riesgo de implantación	15 %	4	4	4	1	2
Independencia del fabricante	10 %	1	2	5	4	5
Total ponderado	**100 %**	**3,10**	**2,85**	**3,35**	**2,35**	**4,05**

Fuente: elaboración propia.

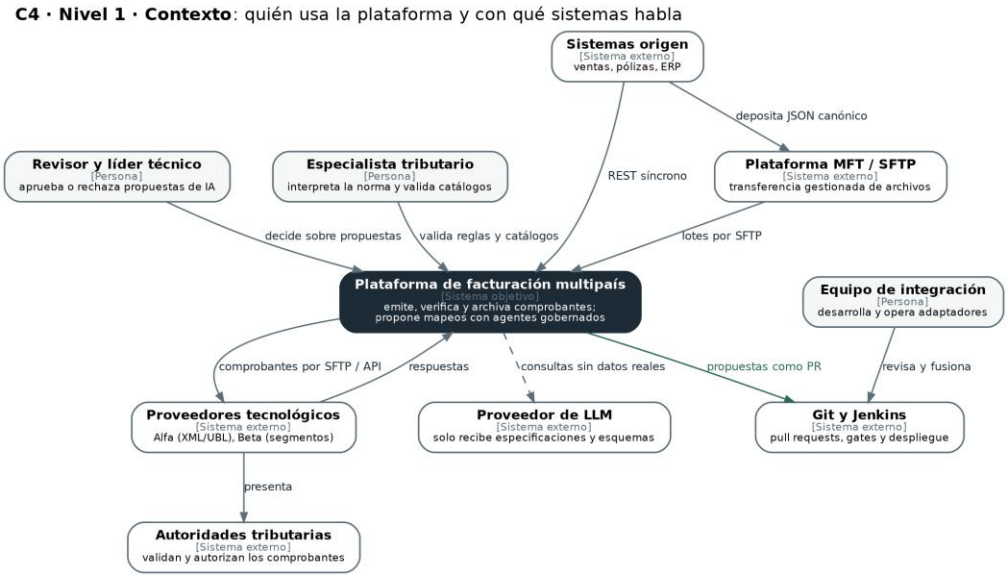
Análisis de sensibilidad. El punto débil de la propuesta es el riesgo de implantación. Si su peso sube y los demás se reducen en proporción, la propuesta sigue ganando mientras ese peso sea menor al 37 %; por encima gana el desarrollo tradicional. La decisión es robusta, pero depende de que el plan reduzca de verdad el riesgo de implantación. Por eso el kit de verificación se construye antes que los agentes: aporta valor por sí solo y, como muestra la prueba de concepto, ya detecta el 86 % de los defectos inyectados.

4.4. Arquitectura de referencia

4.4.1. Contexto del sistema (C4, nivel 1)

La figura 7 muestra quién usa la plataforma y con qué sistemas se comunica. Dos decisiones aparecen ya en este nivel: el proveedor de LLM nunca recibe datos reales, solo especificaciones y esquemas; y todo cambio propuesto por la IA viaja como pull request hacia Git y Jenkins.

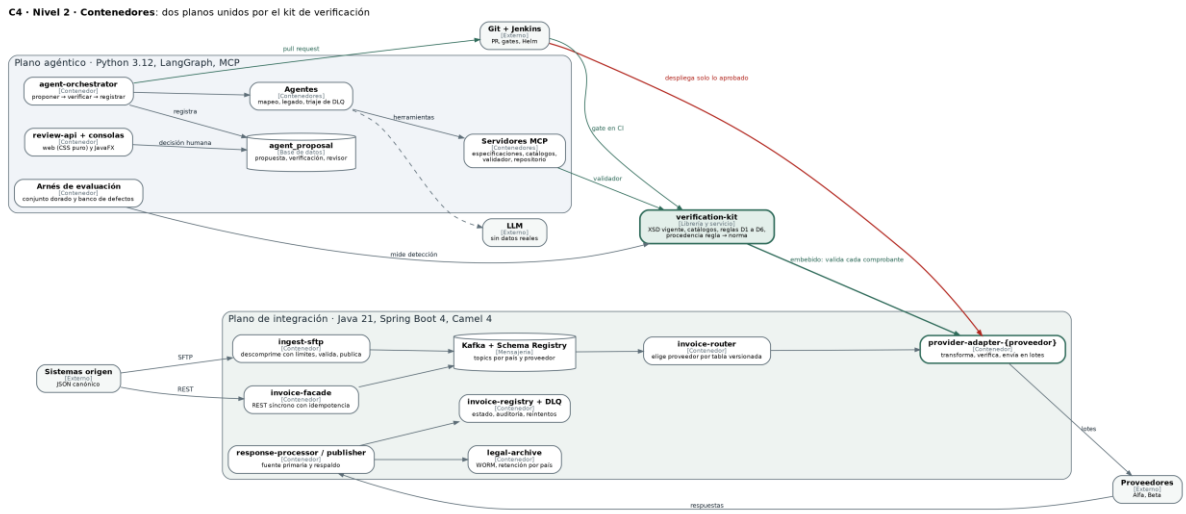
Figura 7. *Diagrama de contexto (C4, nivel 1)*



Fuente: elaboración propia. Elaboración propia con la notación del modelo C4 (Brown, s. f.).

4.4.2. Contenedores (C4, nivel 2)

Figura 8. *Diagrama de contenedores (C4, nivel 2)*



Fuente: elaboración propia. Elaboración propia. En verde, la ruta de verificación; en rojo, el único camino hacia producción.

La arquitectura separa dos planos unidos por el kit de verificación. La tabla siguiente detalla los diez microservicios del plano de integración, con sus entradas y salidas.

Tabla 11. *Microservicios del plano de integración*

#	Microservicio	Responsabilidad	Entrada	Salida
1	ingest-sftp	Polling del MFT, descompresión acotada, validación del canónico con JSON Schema y publicación por país	Archivos JSON/ZIP por SFTP	{pais}.invoice.request.v1
2	invoice-facade	API REST síncrona con clave de idempotencia; invoca el conector síncrono	REST (OpenAPI 3.1)	Respuesta HTTP y evento de trazabilidad
3	invoice-router	Elige el proveedor según país, tipo de documento y tabla de enrutamiento versionada	Topic de solicitud por país	{pais}.{proveedor}.request.v1
4	provider-adapter-{proveedor}	Transforma al formato propietario, verifica con el kit, persiste y envía en lotes	Topic del proveedor	ZIP al proveedor y estado en BD
5	provider-connector-{proveedor}	Conector síncrono del proveedor	Llamada de la fachada	Respuesta del proveedor
6	response-processor	Correlaciona respuestas con fuente primaria y respaldo y actualiza el estado	SFTP de retorno	{pais}.invoice.response.v1
7	response-publisher	Devuelve la respuesta al sistema origen en su formato	Topic de respuesta	SFTP o REST hacia el origen
8	legal-archive	Archiva PDF, XML y constancia con inmutabilidad y retención	Evento de autorización	Almacenamiento WORM
9	invoice-registry	Consulta de estado y auditoría de cada comprobante	REST	Vista de trazabilidad
10	dlq-handler	Reintento controlado y clasificación de errores no recuperables	{pais}.invoice.dlq.v1	Reintento, cuarentena o triaje

Fuente: elaboración propia.

Dos piezas transversales sostienen la reutilización. **verification-kit** es una librería Java embebida en cada adaptador y usada como gate en CI. **adapter-archetype** es un arquetipo Maven que genera un adaptador nuevo con sus pruebas. El plano agéntico, descrito en la tabla siguiente, se comunica con el de integración solo a través del kit y del repositorio.

Tabla 12. *Componentes del plano agéntico*

#	Componente	Responsabilidad
11	agent-orchestrator	Coordina los agentes, mantiene el estado del flujo y exige aprobación humana antes de cualquier cambio
12	mapping-agent	Genera desde la especificación la tabla de mapeo, las plantillas y las pruebas; entrega un pull request, nunca despliega
13	legacy-extraction-agent	Extrae reglas de código heredado con evidencia de archivo y línea
14	dlq-triage-agent	Clasifica los fallos de la DLQ según la taxonomía y propone la corrección en origen
15	mcp-spec-reader	Servidor MCP: búsqueda sobre las especificaciones con sección y hash
16	mcp-catalog	Servidor MCP: catálogos oficiales versionados por país
17	mcp-validator	Servidor MCP: expone el kit para que los agentes validen sus propias propuestas
18	mcp-repo	Servidor MCP: crea ramas y pull requests, sin permiso de merge
19	eval-harness	Evaluación continua contra el conjunto dorado y el banco de defectos

Fuente: elaboración propia.

4.4.3. Stack tecnológico

Tabla 13. *Lenguajes y su justificación*

Lenguaje	Uso	Justificación
Java 21 (LTS)	Microservicios del plano de integración y kit	Hilos virtuales para E/S intensiva; ecosistema maduro de integración
Python 3.12	Agentes, servidores MCP y evaluación	Ecosistema de LLM y MCP más maduro (ADR-03)
HTML y CSS puro, JavaFX	Consolas de revisión humana	Sin frameworks en la web; escritorio nativo para revisores
SQL	Persistencia relacional y migraciones	Trazabilidad transaccional
YAML	Rutas Camel, Kubernetes, Helm, JCasC	Configuración declarativa y versionada

Fuente: elaboración propia.

Tabla 14. *Frameworks, librerías e infraestructura*

Capa	Tecnología
Integración (Java)	Spring Boot 4, Apache Camel 4 (kafka, sftp, zipfile, yaml-dsl), Jackson, Resilience4j, MapStruct
Plano agéntico (Python)	SDK de Anthropic con salida estructurada, SDK de MCP, LangGraph, FastAPI, Pydantic, lxml
Pruebas	JUnit 5, Testcontainers, WireMock, Pact, ArchUnit; pytest e Hypothesis
Calidad y seguridad	SonarQube, OWASP Dependency-Check, Trivy, Checkstyle
Cómputo	Kubernetes administrado con Helm; HPA y NetworkPolicy
Mensajería	Kafka con Schema Registry (compatibilidad BACKWARD), SASL/OAUTHBEARER sobre TLS
Datos	SQL Server con Flyway; PostgreSQL con pgvector para RAG con procedencia
Archivo legal	Almacenamiento de objetos con ciclo de vida e inmutabilidad WORM

Capa	Tecnología
Secretos e identidad	Key Vault y Workload Identity; cero secretos en código o imágenes
Observabilidad	OpenTelemetry, Prometheus, Grafana y alertas
CI/CD	Jenkins como código (JCasC), Helm, promoción SIT → UAT → PROD con aprobación

Fuente: elaboración propia.

4.4.4. Contratos e interfaces

- **Modelo canónico:** InvoiceRequest e InvoiceResponse, registrados en Schema Registry con versionado semántico. La prueba de concepto implementa InvoiceRequest con Pydantic e invariantes de negocio.
- **Convención de topics:** {pais}.invoice.{request|response|dlq}.v{n} y {pais}.{proveedor}.{request|response}.v{n}, particionados por número de comprobante.
- **APIs REST:** OpenAPI 3.1 con enfoque contract-first y pruebas de contrato con Pact.
- **API del kit:** POST /verify?format=xml|seg&mode=RUNTIME|CI devuelve 200 si el documento pasa y 422 con los hallazgos si no; GET /health expone el hash de la especificación cargada.
- **API de revisión:** GET /api/proposals, GET /api/proposals/{id} y POST /api/proposals/{id}/decision, que responde 409 si se intenta aprobar una propuesta que no pasó el kit.
- **Correlación de punta a punta:** un correlationId único viaja por eventos, base de datos y trazas de OpenTelemetry.

4.4.5. Modelo de datos

Tabla 15. *Tablas del núcleo del modelo de datos*

Tabla	Columnas principales	Propósito
authorization_log	id, correlation_id (único), número, tipo, país, proveedor, estado, payload transformado, código y mensaje de error, retry_count, received_at, sent_at, authorized_at, updated_at	Estado y auditoría de cada comprobante; índices por (estado, received_at) y por número
rule_provenance	rule_id, versión, país, proveedor, sección de origen, hash del documento fuente, vigencia	Trazabilidad de cada regla hasta la norma
agent_proposal	id, agente, tipo, spec_sha256, mapeo, resultado de verificación, gate_passed, estado, revisor, comentario, decided_at	Auditoría completa de la IA; implementada en la prueba de concepto

Fuente: elaboración propia.

4.4.6. Seguridad por diseño

- **Mínimo privilegio para la IA:** los agentes solo usan herramientas de lectura y crean pull requests; una NetworkPolicy les impide alcanzar sistemas de producción.

- **Datos:** el LLM recibe especificaciones y esquemas, nunca comprobantes reales; la evaluación usa datos sintéticos.
- **Entradas hostiles:** el kit rechaza DOCTYPE y entidades externas (XXE) y limita el tamaño de los documentos; la ingesta limita la descompresión.
- **Contenedores:** usuario sin privilegios, sistema de archivos de solo lectura y capacidades eliminadas.
- **Cadena de suministro:** escaneo de dependencias, imágenes y secretos en el pipeline; hash del artefacto publicado.

4.5. Decisiones arquitectónicas

Cada decisión se documenta como ADR (Nygard, 2011) con contexto, opciones, decisión, consecuencias y el escenario de calidad que la justifica. La tabla siguiente las resume; el anexo A las presenta completas.

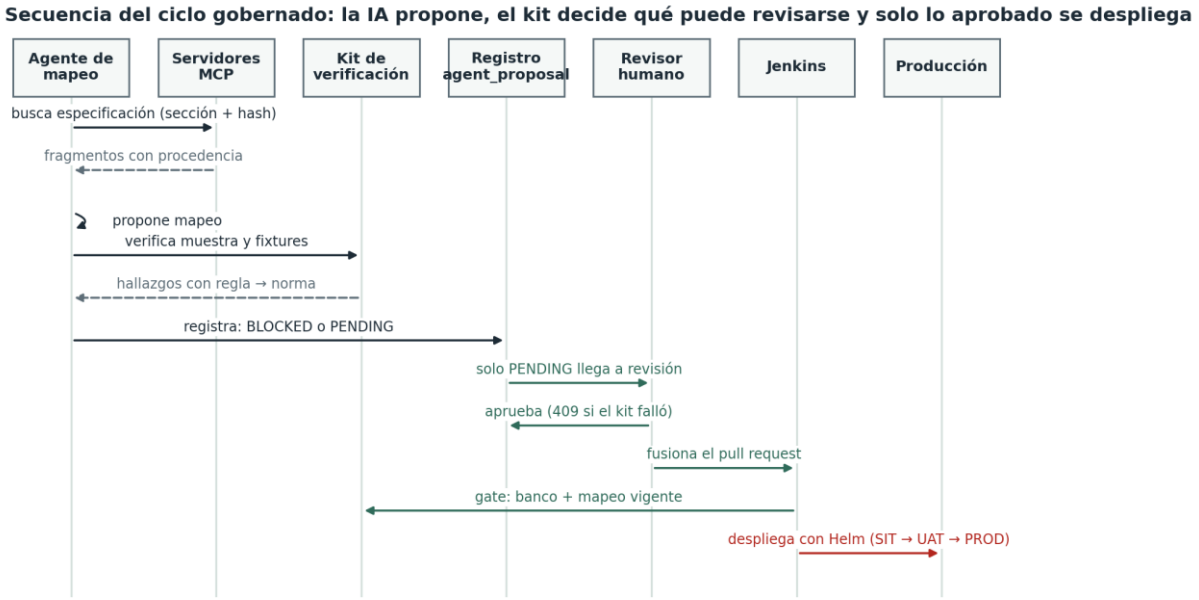
Tabla 16. *Resumen de las decisiones arquitectónicas*

ADR	Decisión	Escenario que la justifica
01	Topics de Kafka por país y proveedor	Fiabilidad: un proveedor caído durante 2 horas no degrada la emisión de los otros países (0 mensajes perdidos, latencia del resto sin cambio).
02	Adaptador por proveedor generado con arquetipo	Mantenibilidad: incorporar un proveedor nuevo parte de un esqueleto que ya compila, pasa el kit y tiene su pipeline.
03	Python para el plano agéntico	Mantenibilidad: el plano agéntico puede cambiar de proveedor de LLM sin tocar el plano de integración.
04	Los agentes solo proponen mediante pull request	Seguridad (responsabilidad): cualquier cambio en producción se puede rastrear hasta su propuesta, su verificación y la persona que la aprobó.
05	Verificación determinista como gate en ejecución y en CI	Adecuación funcional: el 100 % de los comprobantes pasa XSD y reglas antes de salir; ninguna propuesta que falle el kit puede aprobarse.
06	Respuesta con fuente primaria y respaldo	Fiabilidad: ningún comprobante autorizado queda sin estado final tras la reconciliación diaria.
07	Archivo legal asíncrono de mejor esfuerzo con reintento	Disponibilidad: una caída del almacenamiento de 30 minutos no detiene la emisión y se recupera sin pérdida.
08	Schema Registry con compatibilidad BACKWARD	Mantenibilidad: un cambio del canónico no rompe a ningún consumidor desplegado.
09	Migraciones de base de datos con Flyway	Mantenibilidad: cualquier entorno se reconstruye desde cero con el mismo esquema.
10	RAG con procedencia para leer especificaciones	Seguridad (responsabilidad): cada regla y propuesta se rastrea hasta la sección y la versión exacta del documento que la origina.

Fuente: elaboración propia.

Las ADR 04 y 05 son el núcleo de la propuesta: convierten un modelo probabilístico en una pieza aceptable para un dominio regulado. La figura 9 muestra cómo se combinan en el ciclo gobernado.

Figura 9. *Secuencia del ciclo gobernado de una propuesta de IA*



Fuente: elaboración propia. Elaboración propia. Implementada de punta a punta en la prueba de concepto, salvo el despliegue a producción.

4.6. Perfiles de trabajo y matriz de responsabilidades

Tabla 17. *Equipo del proyecto*

Rol	Responsabilidad principal	Capacidad (FTE)
Gerente del proyecto	Plan, presupuesto, riesgos, interesados y accesos	1
Arquitecto de soluciones	Arquitectura, ADR, gobierno de la IA	1
Líder técnico de integración	Diseño detallado, revisión de código y aprobación de propuestas de IA	1
Desarrolladores backend Java (3)	Microservicios, adaptadores y kit	3
Ingeniero de IA/ML	Agentes, servidores MCP y evaluación continua	1
Ingeniero DevOps/SRE	Kubernetes, Jenkins, observabilidad y seguridad	1
QA de automatización	Pruebas de contrato, integración, fixtures y banco de defectos	1
Especialista tributario	Interpretación normativa y catálogos	1

Fuente: elaboración propia.

R = realiza, A = rinde cuentas, C = consultado, I = informado.

Tabla 18. *Matriz RACI*

Actividad	GP	AS	LT	DEV	IA	OPS	QA	TRIB
Arquitectura y ADR	I	A/R	C	C	C	C	I	I
Fundaciones de plataforma	I	C	A	C	I	R	I	I
Plano de integración	I	C	A	R	I	C	C	I
Kit de verificación	I	C	A	R	C	I	R	C
Plano agéntico	I	A	C	I	R	C	C	C
Aprobación de propuestas de IA	I	C	A/R	C	C	I	C	C

Actividad	GP	AS	LT	DEV	IA	OPS	QA	TRIB
Incorporación de un país	A	C	R	R	R	C	R	C
Accesos y credenciales	A/R	I	C	I	I	R	I	I
Plan de pruebas y calidad	I	C	C	R	C	C	A/R	C
Gobierno y riesgos de IA	A	R	C	I	R	C	C	C
Presupuesto y seguimiento	A/R	C	C	I	I	I	I	I

Fuente: elaboración propia.

4.7. Planificación de tareas, esfuerzos y cronograma

El proyecto se planifica en seis fases con cinco hitos de control, un equipo de diez personas en ocho roles y un presupuesto que incluye el consumo de LLM como partida propia. Todas las cifras de este capítulo salen de un modelo reproducible desarrollado para este trabajo (anexo B): estimaciones de tres puntos, nivelación de recursos y 10.000 iteraciones de Monte Carlo. Los supuestos están declarados en el anexo B.

4.7.1. Fases e hitos de control

Tabla 19. *Fases e hitos de control del proyecto*

Fase	Contenido	Hito de salida (criterio verificable)
F1 Fundaciones	Planificación, accesos, Kubernetes, CI/CD, Kafka, observabilidad	G0: entornos SIT y UAT operativos con el pipeline en verde
F2 Kit de verificación	XSD por país, catálogos versionados, reglas D1 a D6, gate en CI	G1: kit como gate en CI y banco con ≥ 80 % de detección en ejecución
F3 Primer país	Plano de integración y primer país sin agentes (línea base)	G2: primer país autorizado en producción
F4 Plano agéntico	Servidores MCP, agentes, arnés y conjunto dorado	G3 (decisión de continuar): los agentes superan el umbral del conjunto dorado
F5 Segundo país asistido	Incorporación con agentes en condiciones comparables	G4: segundo país autorizado en producción
F6 Medición y cierre	Línea base, banco de defectos, informe y transferencia	G5: informe de resultados aprobado

Fuente: elaboración propia.

El orden de las fases no es arbitrario. El kit llega antes que los agentes porque aporta valor por sí solo y reduce el riesgo de implantación, el punto débil señalado en la matriz de decisión. El primer país se incorpora sin agentes para medir la línea base; el segundo, con agentes, para medir la diferencia en condiciones comparables. **G3 es un punto de decisión real:** si los agentes no alcanzan el umbral, la fase 5 continúa sin ellos y el proyecto conserva el valor del kit y del arquetipo.

4.7.2. Estructura de desglose del trabajo y estimaciones

Cada paquete de trabajo se estima con tres valores en semanas: optimista (o), más probable (m) y pesimista (p). La duración esperada sigue la aproximación PERT (Malcolm et al., 1959):

$$t_e = (o + 4m + p) / 6$$

El esfuerzo de cada paquete es su duración esperada por la suma de las dedicaciones de los roles asignados. A estas se suma una dedicación continua de gerencia (0,5), arquitectura (0,5), liderazgo técnico (1,0) y especialista tributario (0,25) durante todo el proyecto.

Tabla 20. *Estructura de desglose del trabajo con estimaciones de tres puntos*

ID	Paquete de trabajo	Fase	o	m	p	t_e	Esfuerzo (p-s)	Depende de
1.1	Planificación y línea base	F1	2	3	4	3,0	3,0	
1.3	Accesos, credenciales y llaves	F1	4	6	12	6,7	2,0	1.1 (recurso: 2.1)
2.1	Kubernetes, Helm y CI/CD	F1	3	4	6	4,2	4,2	1.1
2.2	Kafka y Schema Registry	F1	2	3	5	3,2	4,8	2.1
2.3	Observabilidad, secretos e identidad	F1	2	3	4	3,0	1,5	2.1 (recurso: 1.3)
3.1	Ingesta y fachada síncrona	F3	3	4	6	4,2	5,4	2.2
3.2	Enrutador y tabla de enrutamiento	F3	1	2	3	2,0	2,0	2.2
3.3	Arquetipo de adaptador	F3	2	3	5	3,2	3,2	3.2, 4.4
3.4	Respuestas y archivo legal	F3	3	4	6	4,2	5,4	3.1
3.5	Registro, DLQ y reintentos	F3	2	3	4	3,0	1,5	3.1
4.1	Validación XSD/UBL por país	F2	1	2	3	2,0	2,0	1.1
4.2	Catálogos oficiales versionados	F2	2	3	4	3,0	2,2	4.1
4.3	Reglas D1 a D6 con procedencia	F2	3	4	7	4,3	7,6	4.2
4.4	Gate del kit en CI	F2	1	2	3	2,0	2,0	4.3, 2.1
5.1	Servidores MCP	F4	2	3	5	3,2	3,2	4.4
5.2	Agente de mapeo	F4	4	6	10	6,3	6,3	5.1, 5.5
5.3	Agente de extracción de legado	F4	3	4	7	4,3	4,3	5.1 (recurso: 5.2)
5.4	Agente de triaje de DLQ	F4	2	3	5	3,2	1,6	5.1, 3.5
5.5	Arnés de evaluación y conjunto dorado	F4	2	3	5	3,2	3,2	4.3 (recurso: 3.3)
6.1	Primer país sin agentes	F3	6	8	14	8,7	27,7	3.3, 3.4, 1.3 (recurso: 5.5)
6.2	Segundo país asistido por agentes	F5	4	6	11	6,5	20,8	6.1, 5.2, 7.2
7.1	Línea base del proceso manual	F6	1	2	3	2,0	1,0	6.1
7.2	Banco de defectos por inyección	F6	1	2	3	2,0	1,6	5.5 (recurso: 6.1)
7.3	Informe de resultados y cierre	F6	2	2	4	2,3	2,3	6.2, 7.1

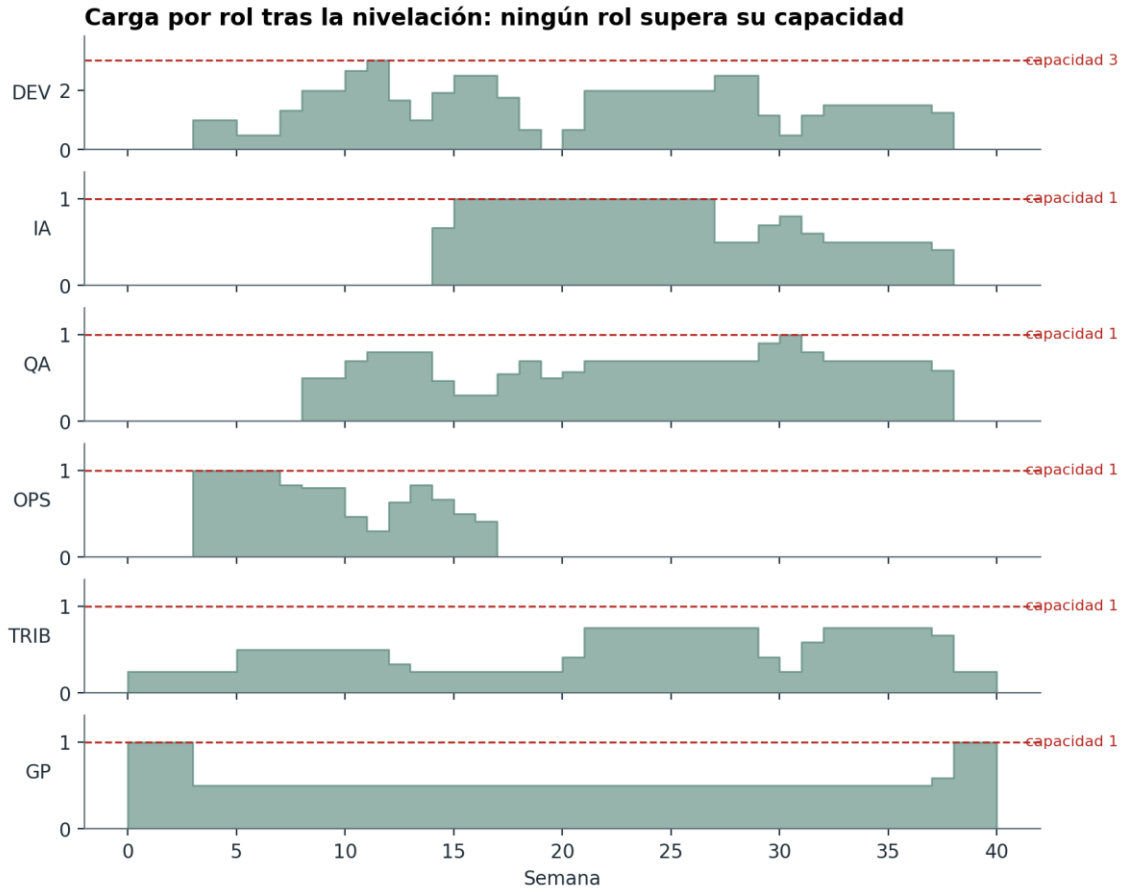
Fuente: elaboración propia. O, m, p y t_e en semanas; esfuerzo en personas-semana (p-s). Las dependencias de recurso las añade la nivelación (apartado 4.7.3).

4.7.3. Nivelación de recursos

Un cronograma calculado solo con dependencias técnicas sobreasignaba al ingeniero de IA (2,0 FTE en su pico, con capacidad de 1), a DevOps y a QA. Un plan que ignora la capacidad del equipo no es ejecutable. El modelo aplica un esquema serial de generación de cronogramas (Kolisch, 1996): programa los paquetes en orden de prioridad, según su inicio tardío, y retrasa cada uno hasta que ningún rol supere su capacidad en ninguna semana. Cada retraso por recurso se registra como una dependencia fin-inicio explícita, de modo que la simulación posterior respeta la capacidad del equipo.

La nivelación añadió 6 dependencias de recurso: 2.1 → 1.3; 3.3 → 5.5; 5.5 → 6.1; 6.1 → 7.2; 5.2 → 5.3; 1.3 → 2.3. La duración esperada pasó de unas 36 a 40,2 semanas. Ese es el costo real de un equipo de diez personas, y la figura 10 confirma que, tras nivelar, ningún rol supera su capacidad.

Figura 10. *Carga semanal por rol tras la nivelación*

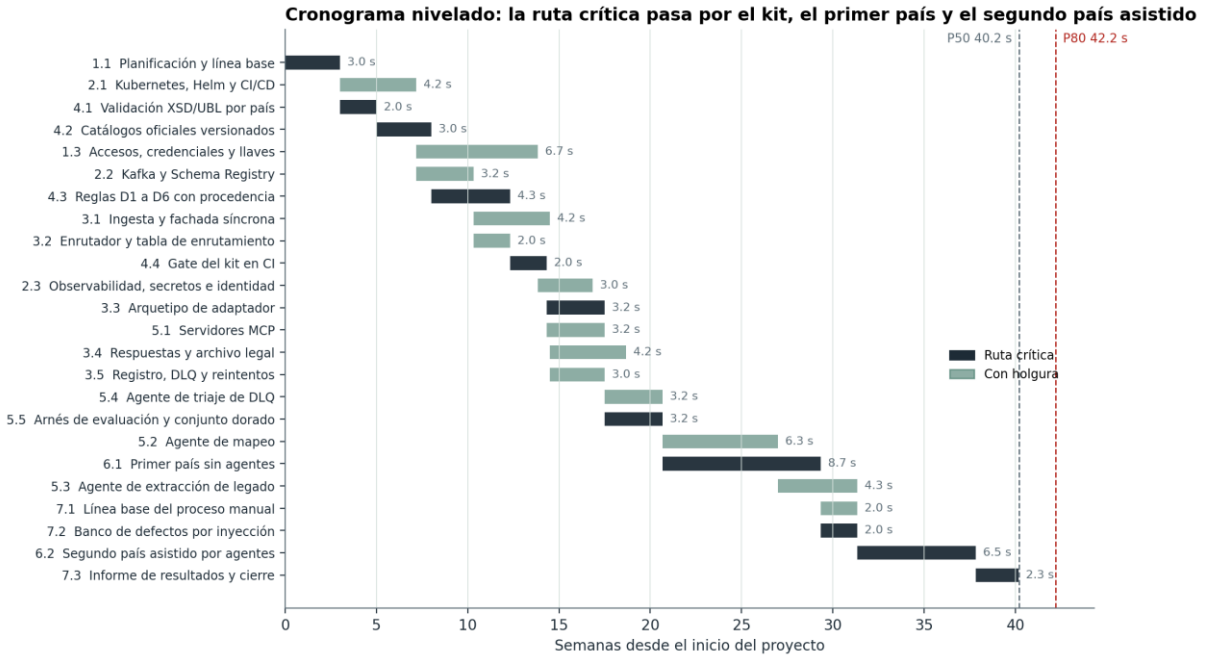


Fuente: elaboración propia. Salida del modelo de planificación. La línea discontinua es la capacidad del rol.

4.7.4. Cronograma nivelado

La ruta crítica determinística es 1.1 → 4.1 → 4.2 → 4.3 → 4.4 → 3.3 → 5.5 → 6.1 → 7.2 → 6.2 → 7.3. Pasa por el kit de verificación (4.1 a 4.4), el arquetipo (3.3), el primer país (6.1) y el segundo país asistido (6.2). Por eso el plan concentra en esos paquetes a las personas más experimentadas y los vigila semanalmente.

Figura 11. *Cronograma nivelado con ruta crítica, P50 y P80*



Fuente: elaboración propia. Salida del modelo de planificación con duraciones esperadas. En oscuro, la ruta crítica.

4.8. Presupuesto

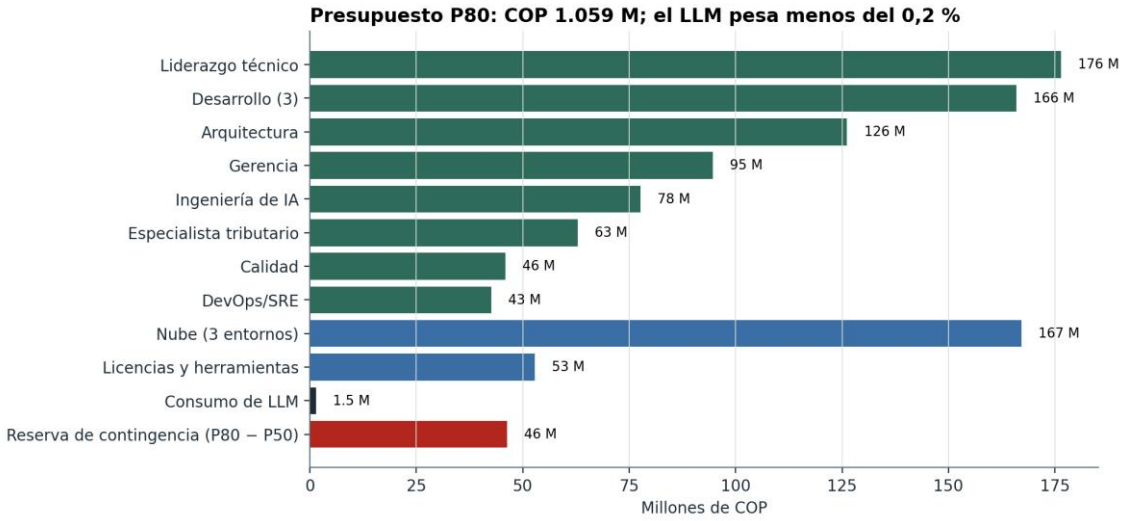
El presupuesto tiene cuatro partidas: personas (dedicación por rol × semanas × costo empresa), nube, licencias y herramientas, y consumo de LLM. La tabla siguiente muestra el presupuesto con duraciones esperadas; la reserva de contingencia lo lleva al P80.

Tabla 21. *Presupuesto por partida (millones de COP)*

Partida	Millones de COP	Participación
Líder técnico de integración	176,4	17,4 %
Desarrolladores backend Java (3)	165,9	16,4 %
Arquitecto de soluciones	126,2	12,4 %
Gerente del proyecto	94,6	9,3 %
Ingeniero de IA/ML	77,7	7,7 %
Especialista tributario	62,9	6,2 %
QA de automatización	45,9	4,5 %
Ingeniero DevOps/SRE	42,6	4,2 %
Subtotal personas	792,3	78,2 %
Nube (tres entornos)	167,1	16,5 %
Licencias y herramientas	52,9	5,2 %
Consumo de LLM	1,5	0,15 %
Total con duraciones esperadas	**1.013,7**	**100 %**
Reserva de contingencia (P80 – P50)	46,3	—
Presupuesto P80 (simulación)	**1.059,2**	—

Fuente: elaboración propia. Costos de personal como costo empresa mensual supuesto (anexo B); TRM supuesta de 4.000 COP/USD.

Figura 12. *Presupuesto por partida*



Fuente: elaboración propia. Salida del modelo de planificación.

El consumo de LLM se estima por agente con la expresión siguiente, donde N es el número de ejecuciones, T los tokens de entrada y salida y p el precio por token:

$$C_LLM = \Sigma_a N_a \cdot (T_a,entrada \cdot p_entrada + T_a,salida \cdot p_salida)$$

Tabla 22. *Consumo estimado de LLM por agente*

Agente	Ejecuciones	Tokens de entrada	Tokens de salida	USD	COP
Mapeo	60	60.000	8.000	18,00	72.000
Legado	250	30.000	4.000	37,50	150.000
Triaje	6.000	6.000	1.000	198,00	792.000
Arnes	400	60.000	8.000	120,00	480.000
Total				**373,50**	**1.494.000**

Fuente: elaboración propia.

El hallazgo es contraintuitivo: el LLM representa menos del 0,2 % del presupuesto. El costo de la IA no está en los tokens, sino en las personas que diseñan la verificación, revisan las propuestas y mantienen el conjunto dorado. Por eso el plan invierte en el kit y en la revisión, no en ahorrar tokens.

4.8.1. Referentes salariales y sensibilidad del presupuesto

Las tarifas por rol son supuestos del autor, calibrados contra referentes públicos del mercado colombiano. El costo empresa mensual de cada rol resulta de multiplicar un salario base por un factor de 1,5, que cubre prestaciones sociales, seguridad social y aportes parafiscales; las fuentes del mercado sitúan ese costo adicional entre el 38 % y el 58 % sobre el salario (Buk, 2026). La tabla siguiente compara el salario base implícito de cada rol con los rangos publicados. Cinco roles tienen un referente público y quedan dentro de su rango; los

de IA, QA y especialista tributario no tienen una cifra pública verificada, por lo que se declaran como supuestos.

Tabla 23. *Salario base implícito por rol y referentes públicos del mercado colombiano*

Rol	Salario base implícito (COP al mes)	Referente público	Estado
Gerente del proyecto	12,0 M	Gerente de proyectos de TI: 7 a 12 M (Colombia Move, 2026)	Dentro del rango, en el extremo superior
Arquitecto de soluciones	16,0 M	Desarrollador senior o arquitecto: 8 a 15 M o más (Colombia Move, 2026); perfiles senior de software hasta 23 M (Michael Page, 2026, como se citó en Portafolio, 2026, y en Valora Analitik, 2026)	Dentro del rango de perfiles senior
Líder técnico de integración	12,7 M	Mismo referente de desarrollador senior o arquitecto	Dentro del rango
Desarrolladores backend Java (3)	8,7 M	Desarrollador senior: 8 a 15 M o más (Colombia Move, 2026)	Dentro del rango, en el extremo inferior
Ingeniero DevOps/SRE	12,0 M	Ingenieros DevOps: 12 a 25 M (Robert Half Technology, 2024, como se citó en La FM, 2024)	Dentro del rango, en el extremo inferior
Ingeniero de IA/ML	12,0 M	Perfiles de aprendizaje automático entre los mejor pagados, sin cifra pública verificable (Michael Page, 2026, como se citó en Portafolio, 2026)	Supuesto del autor
QA de automatización	6,7 M	Sin referente público verificado	Supuesto del autor
Especialista tributario	9,3 M	Sin referente público verificado	Supuesto del autor

Fuente: elaboración propia. M = millones de pesos colombianos. El salario base implícito es el costo empresa mensual dividido por el factor de 1,5.

Para que la conclusión no dependa de un único conjunto de tarifas, el modelo se ejecutó también con tarifas un 20 % menores y un 25 % mayores. El calendario no cambia, porque no depende de las tarifas; solo se mueve el costo, y el proyecto sigue siendo viable en los tres escenarios. El punto de equilibrio, expresado en esfuerzo, tampoco depende de ellas.

Tabla 24. *Sensibilidad del costo del proyecto a las tarifas por rol*

Escenario	Tarifas por rol	Costo P50 (millones de COP)	Costo P80 (millones de COP)
Bajo	20 % menos	855	894
Base	**Calibradas (tabla anterior)**	**1.013**	**1.059**
Alto	25 % más	1.211	1.266

Fuente: elaboración propia. Salida del modelo de planificación con 10.000 iteraciones por escenario.

4.9. Análisis de riesgos

Los riesgos de IA se asignan a las funciones del marco del NIST: gobernar (G), mapear (Ma), medir (Me) y gestionar (Ge) (NIST, 2023). Probabilidad (P) e impacto (I): A = alto, M = medio, B = bajo.

Tabla 25. *Registro de riesgos*

ID	Riesgo	P	I	Respuesta	NIST	Dueño
R1	El agente propone mapeos incorrectos pero plausibles	A	A	Kit obligatorio, revisión humana y banco de defectos (comprobado en la PoC)	Me, Ge	LT
R2	Deriva del modelo o cambio de versión del proveedor de LLM	M	A	Versión fijada y evaluación continua contra el conjunto dorado	Me	IA
R3	Exposición de datos tributarios en las consultas al LLM	M	A	Solo especificaciones y esquemas; datos sintéticos; acuerdos de no retención	G, Ge	AS
R4	Cambio normativo durante el proyecto	A	M	Catálogos versionados y reglas con procedencia y hash	Ma	TRIB
R5	Retraso en credenciales, llaves y accesos	A	A	Gestión desde la fase 1 y trabajo en paralelo	—	GP
R6	Confianza excesiva de los revisores en la IA	M	A	Gate que impide aprobar lo bloqueado; auditoría por muestreo	G, Ge	QA
R7	Consumo de LLM por encima de lo presupuestado	B	B	Límites por ejecución; el LLM pesa menos del 0,2 %	Ge	IA
R8	Dependencia de un único experto tributario	M	M	Reglas como código con procedencia; transferencia de conocimiento	G	GP
R9	QA como cuello de botella del calendario	A	M	Segunda persona de QA en F2 a F4 (escenario del apartado 5.6)	—	GP

Fuente: elaboración propia.

4.10. Plan de calidad, integración y despliegue continuos y gobierno del proyecto

4.10.1. Plan de calidad y de pruebas

La calidad se asegura con controles automáticos en el pipeline: análisis estático con quality gate, escaneo de dependencias, imágenes y secretos, pruebas de contrato, reglas de arquitectura y, como control distintivo, el kit de verificación con el banco de defectos. Las pruebas se organizan en cinco niveles:

1. Unitarias, incluidas pruebas basadas en propiedades para las invariantes de D2.
2. De integración con Kafka, base de datos y SFTP reales en contenedores.
3. De contrato entre servicios y con proveedores simulados.
4. De carga para el escenario de 200.000 comprobantes en un día pico.
5. De evaluación de agentes con el conjunto dorado y el banco de defectos por inyección.

4.10.2. Integración y despliegue continuos con Jenkins

Jenkins se instala como código: una imagen con las herramientas del pipeline, los plugins declarados y la configuración JCasC con usuarios, permisos, credenciales y el job del pipeline. Cada etapa del Jenkinsfile invoca el mismo script que se ejecuta en local, de modo que lo que pasa en el portátil del desarrollador pasa igual en el servidor.

Tabla 26. *Etapas del pipeline de Jenkins*

Etapa	Qué hace	Detiene el pipeline si…
Compilar	Compila el kit con -Xlint:all -Werror y la consola JavaFX	Hay errores o advertencias de compilación
Pruebas (en paralelo)	JUnit 5 del kit y pytest con Hypothesis del plano agéntico; publica reportes JUnit	Falla una prueba
Gate · banco de defectos	Ejecuta los 22 mutantes sobre 300 comprobantes y 12 fixtures	CI no detecta el 100 %, ejecución < 80 % o hay falsos positivos
Gate · mapeo vigente	Verifica el mapeo aprobado en modo CI sobre fixtures y una muestra	Algún documento tiene hallazgos
Seguridad	Trivy sobre el repositorio: vulnerabilidades, secretos y configuración	Hallazgos altos o críticos
Empaquetar	JAR del kit con su hash y prueba de humo del servicio HTTP	/verify no responde 200 y 422 como se espera
Imágenes	Construye, escanea y publica las imágenes	Vulnerabilidades críticas
Desplegar SIT	Helm upgrade --install y prueba de humo en el clúster	El despliegue o la prueba fallan
Aprobación y UAT/PROD	input con aprobadores definidos; Helm con --atomic	Nadie aprueba o el despliegue falla (rollback automático)

Fuente: elaboración propia.

El chart de Helm despliega el kit con autoescalado horizontal y el plano agéntico con un volumen persistente para el registro de propuestas. Una NetworkPolicy limita la salida del plano agéntico al kit y a HTTPS hacia el proveedor del LLM. Los contenedores se ejecutan sin privilegios y con sistema de archivos de solo lectura.

4.10.3. Gobierno del proyecto

El seguimiento semanal revisa el avance sobre la ruta crítica y el índice de criticidad; el quincenal, los riesgos y el consumo de la reserva. Cada hito G0 a G5 es una revisión formal con criterio verificable, y G3 incluye la decisión explícita de continuar o no con los agentes. Las métricas del capítulo 3 se publican en un tablero desde la fase F2.

4.11. Evaluación de la solución

La evaluación de la solución responde a cuatro preguntas: qué se mide, cuándo, quién lo mide y qué resultado se considera un éxito. Los indicadores derivan de las métricas del enfoque objetivo, pregunta y métrica presentado en la tabla 6, y se miden en las fases del proyecto indicadas en la tabla siguiente. La evaluación no espera al final: el kit se evalúa en cada cambio de mapeo desde la fase F2, el agente de mapeo se mide con el arnés de evaluación en la fase F4 y el esfuerzo por país se contrasta con la línea base manual en la fase F5.

Tabla 27. *Plan de evaluación de la solución*

Indicador	Cuándo se mide	Quién	Criterio de éxito
Tasa de detección en ejecución	Cada cambio de mapeo, en el gate de integración continua	Equipo de integración y QA	Al menos 80 % de los defectos inyectados

Indicador	Cuándo se mide	Quién	Criterio de éxito
Falsos positivos del kit	Cada ejecución del banco y del conjunto dorado	QA	Cero sobre salidas correctas
Comprobantes verificados antes del envío	Continuo, en producción	Equipo de integración	100 % de los comprobantes pasan el kit antes de salir al proveedor
Esfuerzo por país o proveedor nuevo	Al cerrar cada incorporación, desde la fase F5	Gerente del proyecto	Ahorro frente a la línea base manual de 27,7 personas-semana
Exactitud del agente de mapeo	Con el arnés de evaluación, en la fase F4	Ingeniero de IA	Medida y comparada con la línea base heurística
Plazo y costo del proyecto	En cada hito de control, con el seguimiento sobre la ruta crítica	Gerente del proyecto	Plazo dentro del P80 y costo dentro del presupuesto con reserva
Riesgos materializados	Revisión quincenal del registro de riesgos	Gerente del proyecto y líder técnico	Ningún riesgo de impacto alto sin plan de contingencia activo

Fuente: elaboración propia.

Los factores de éxito son tres. El primero es que el kit supere la meta de detección sin falsos positivos, porque de ello depende que la revisión humana pueda confiar en lo que recibe. El segundo es que el esfuerzo asistido por agentes sea efectivamente menor que el manual, lo que la planificación formula como hipótesis y la fase F5 comprueba con la primera incorporación real. El tercero es que el proyecto se mantenga dentro del plazo y del presupuesto del percentil 80, con la reserva de contingencia como margen.

5. Resultados obtenidos

Este capítulo presenta los resultados del trabajo organizados por la evidencia que los respalda. Primero, la prueba de concepto y su banco de defectos, que validan el diseño del kit de verificación (apartados 5.1 a 5.5). Después, los resultados de la planificación, con la simulación de Monte Carlo y el punto de equilibrio (5.6), y las amenazas a la validez de lo medido (5.7). Al final, el apartado 5.8 contrasta los resultados con cada objetivo específico.

5.1. Prueba de concepto: alcance, implementación y pruebas

Para no usar información de ninguna organización, se construyó una especificación sintética, el **Perfil Alfa v2.1**, que imita la estructura de los anexos técnicos latinoamericanos basados en UBL 2.1: secciones numeradas, catálogos versionados, reglas de validación y elementos retirados por versión. Se añadió un segundo formato, el del **proveedor Beta**, de texto por segmentos con separador reservado, para cubrir el defecto D6. El generador produce comprobantes sintéticos reproducibles con semilla fija, y 12 fixtures curados cubren cada tipo de documento, cada tipo de identificación, cada tributo y datos adversarios.

5.1.1. Implementación

Tabla 28. *Componentes implementados en la prueba de concepto*

Componente	Tecnología	Contenido	Líneas
verification-kit	Java 21, solo JDK	XSD, reglas D1 a D6 con procedencia, fixtures, CLI y servidor HTTP	725
agentic-plane	Python 3.12, Pydantic, lxml, SDK de MCP 2.x, FastAPI	Canónico, transformadores, agente de mapeo, orquestador, registro, servidor MCP, API, banco	997
review-console-web	HTML y CSS puro	Consola de revisión sin frameworks	321
review-console-javafx	JavaFX con hoja CSS	Consola de escritorio contra la misma API	349
Pruebas	JUnit 5, pytest, Hypothesis	32 pruebas automatizadas	223
ci y deploy	Bash, Jenkins (JCasC), Docker, Helm	Pipeline, imágenes, chart y políticas de red	408
planning	Python, NumPy	PERT, nivelación de recursos, Monte Carlo y escenarios	171

Fuente: elaboración propia. Líneas de código no vacías.

El kit no tiene dependencias externas, por decisión deliberada: el mismo JAR corre en un adaptador, en Jenkins y detrás del servidor MCP. Rechaza DOCTYPE para prevenir XXE, limita el tamaño de los documentos y devuelve código de salida 2 cuando hay hallazgos, lo que lo convierte en un gate directo para cualquier pipeline.

5.1.2. Pruebas automatizadas

Tabla 29. *Suites de pruebas y resultados*

Suite	Herramienta	Pruebas	Qué verifican	Resultado
Kit de verificación	JUnit 5	17	Cada regla D1 a D6, procedencia, cero falsos positivos, XXE, segmentos y el caso que solo detecta el fixture	17/17
Modelo canónico	pytest + Hypothesis	3	Invariantes en comprobantes generados al azar y cobertura de los fixtures	3/3
Integración Python–Java	pytest	3	Mapeo dorado sin falsos positivos y banco completo con CI al 100 %	3/3
Agente de mapeo	pytest	3	Exactitud mínima, bloqueo por el kit y filtrado de alucinaciones del LLM	3/3
API de revisión	pytest + TestClient	3	409 al aprobar lo bloqueado, decisión única, validación del revisor	3/3
Servidor MCP	pytest	3	Herramientas expuestas, procedencia en la búsqueda y validador	3/3

Fuente: elaboración propia.

5.2. Banco de defectos por inyección

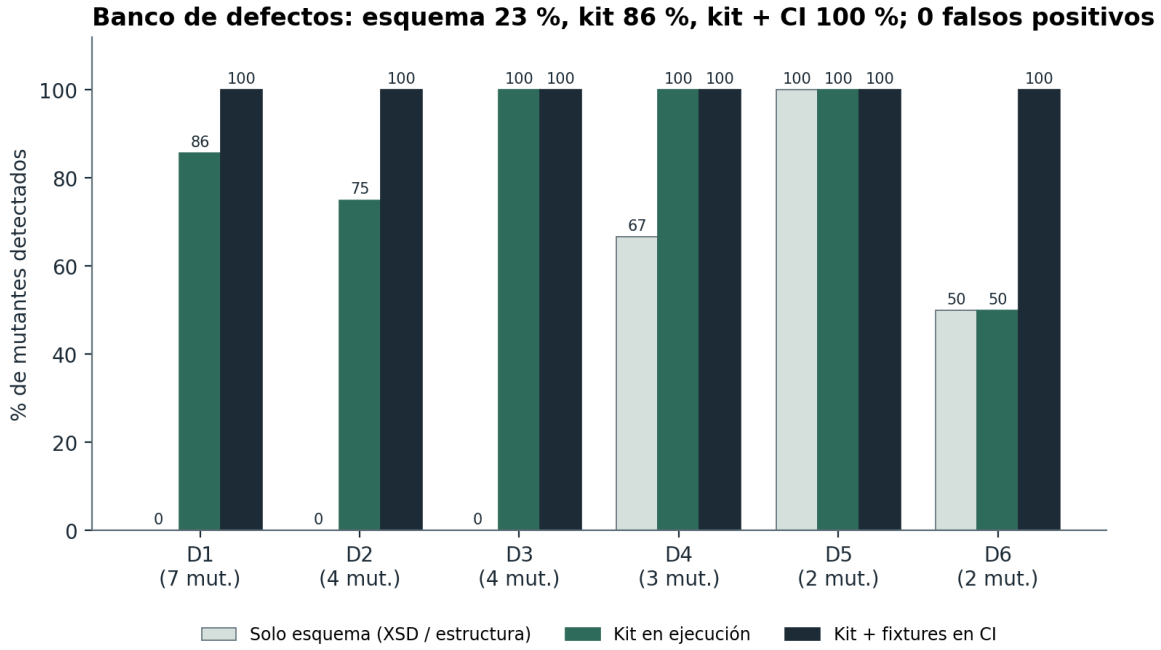
El banco aplica 22 mutantes, de los seis tipos, sobre 300 comprobantes sintéticos y 12 fixtures. En total son 14.688 verificaciones, que el kit resolvió en 14,4 segundos en un solo proceso, unas 1.022 por segundo incluido el arranque de la JVM. Un mutante se considera detectado si el mecanismo rechaza al menos un documento afectado.

Tabla 30. *Mutantes detectados por tipo y mecanismo*

Tipo	Mutantes	Solo esquema	Kit en ejecución	Kit + CI
D1	7	0 %	86 %	100 %
D2	4	0 %	75 %	100 %
D3	4	0 %	100 %	100 %
D4	3	67 %	100 %	100 %
D5	2	100 %	100 %	100 %
D6	2	50 %	50 %	100 %
Total	**22**	**23 %**	**86 %**	**100 %**

Fuente: elaboración propia. Falsos positivos sobre 1.224 verificaciones de salidas correctas: 0 en los tres mecanismos.

Figura 13. *Detección de defectos por tipo y mecanismo*



Fuente: elaboración propia. Resultados reales de la prueba de concepto (anexo C).

Los resultados confirman la hipótesis de diseño con datos. La validación de esquema, la práctica habitual, detecta solo el 23 % de los mutantes: falla con todos los D1, D2 y D3, porque en UBL los códigos y literales son cadenas libres. Las reglas con procedencia elevan la detección al 86 %, sin un solo falso positivo. Los tres mutantes que escapan en ejecución explican por qué el gate debe ser doble:

- **D1-e.** Pasaporte codificado como documento de extranjero (A3); formatos idénticos. Los formatos de ambos documentos son idénticos, así que ninguna regla puede distinguirlos. Solo lo detecta la comparación con fixtures validados por personas, que incluyen datos adversarios.
- **D2-a.** Cantidad y precio unitario intercambiados (producto idéntico). El producto cantidad × precio no cambia, así que todas las invariantes aritméticas se cumplen. Solo lo detecta la comparación con fixtures validados por personas, que incluyen datos adversarios.
- **D6-b.** Nombre del adquirente sin sanear el separador reservado. Ningún comprobante de la muestra contenía el separador en el nombre: el defecto existe pero los datos normales no lo activan. Solo lo detecta la comparación con fixtures validados por personas, que incluyen datos adversarios.

La conclusión práctica es que ni el esquema ni las reglas bastan por separado: el kit en ejecución protege la producción, y los fixtures en CI cierran los huecos que ninguna regla puede ver. Es exactamente la ADR-05.

5.3. Agente de mapeo y ciclo gobernado

El agente de mapeo tiene dos motores detrás del mismo contrato. El motor heurístico, por similitud léxica con un glosario mínimo, sirve de línea base determinista. El motor LLM usa el SDK de Anthropic con salida JSON y filtra los campos y fuentes inexistentes, lo que mitiga las alucinaciones estructurales; se probó con un cliente simulado, porque el entorno de desarrollo del prototipo no tenía credenciales de API. La medición con un LLM real es una tarea del arnés de evaluación en la fase F4.

El ciclo gobernado se ejecutó de punta a punta con el motor heurístico:

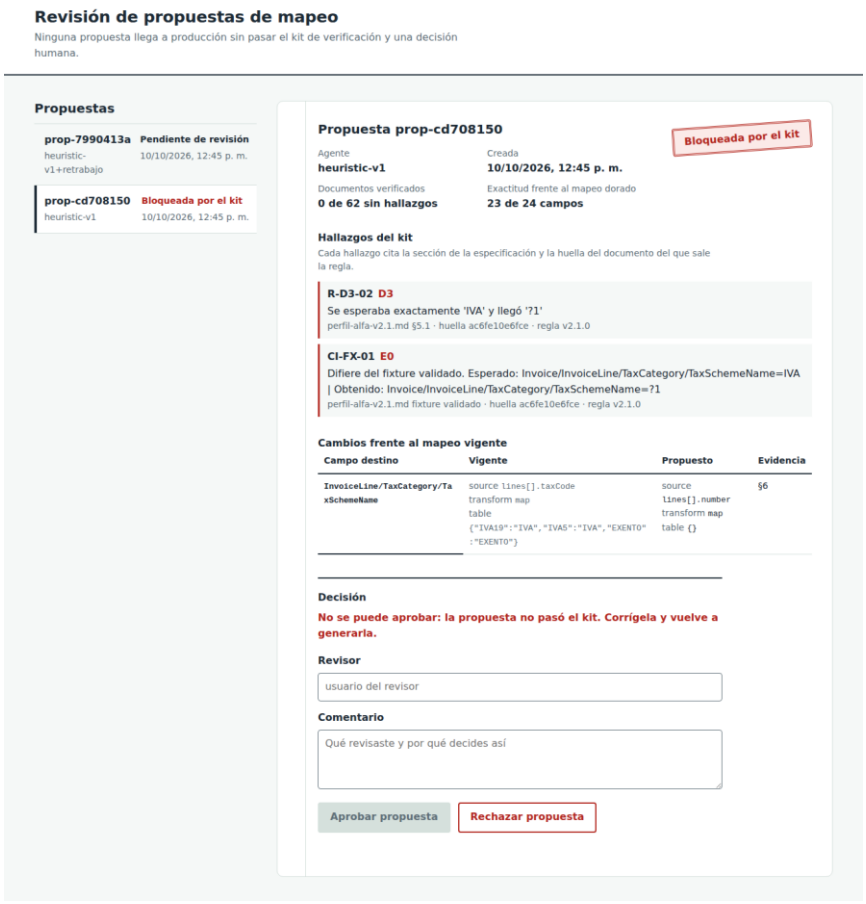
1. El agente propuso los 24 campos del perfil y acertó 23, una exactitud del 95,8 % frente al mapeo dorado.
2. Su único error fue mapear el nombre del esquema tributario al número de línea. El kit bloqueó la propuesta en los 62 documentos verificados con la regla R-D3-02, que cita la §5.1 de la especificación y su hash.
3. La propuesta quedó en estado BLOCKED_BY_GATE y la API impide aprobarla (HTTP 409).
4. La propuesta corregida pasó el kit en los 62 documentos y quedó pendiente de decisión humana.

El caso ilustra el valor del diseño. Una exactitud del 95,8 % parece excelente, pero el campo equivocado habría producido un rechazo en cada comprobante. El kit convierte un error silencioso en un hallazgo explicado y trazable antes de que llegue a una persona.

5.4. Consolas de revisión

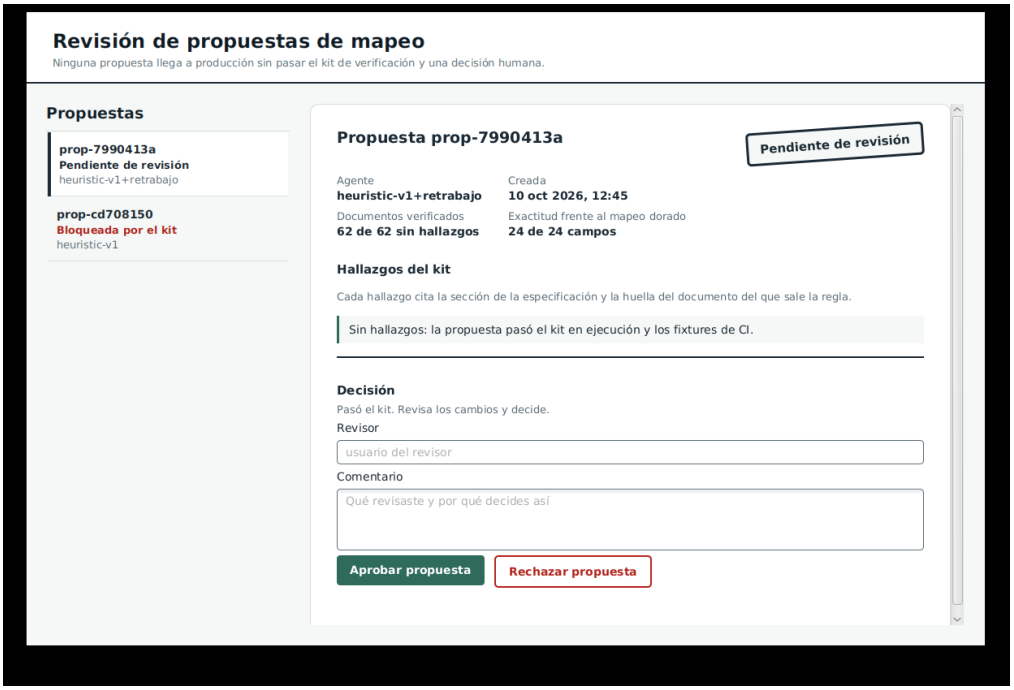
La figura 14 muestra la consola web, escrita con HTML y CSS puro, sin frameworks; el JavaScript se limita a pedir datos a la API. El sello indica el estado del gate, y cada hallazgo muestra la regla, la sección de la norma y la huella del documento. El botón de aprobar está deshabilitado mientras la propuesta no pase el kit. La figura 15 muestra la consola de escritorio en JavaFX, con la misma identidad visual definida en una hoja CSS de JavaFX, ejecutándose contra la misma API.

Figura 14. *Consola web de revisión con una propuesta bloqueada por el kit*



Fuente: elaboración propia. Captura de la prueba de concepto en ejecución.

Figura 15. *Consola de escritorio JavaFX con una propuesta pendiente de revisión*



Fuente: elaboración propia. Captura de la aplicación JavaFX ejecutada contra la API de revisión.

5.5. Pipeline, despliegue y madurez alcanzada

El script de etapas del pipeline se ejecutó completo: compilación con advertencias tratadas como errores, 17 pruebas JUnit, 15 pruebas pytest, gate del banco de defectos, gate del mapeo vigente (112 documentos sin hallazgos), empaquetado del JAR con su hash y prueba de humo del servicio HTTP, que respondió 200 a un comprobante válido y 422 a uno inválido. El log completo de la ejecución se conserva como evidencia del trabajo. Jenkins se levanta con docker compose a partir de la configuración como código incluida, y ejecuta esas mismas etapas más la construcción de imágenes y el despliegue con Helm.

5.5.1. Madurez alcanzada

El kit de verificación queda en TRL 4: validado en laboratorio con datos sintéticos y un banco de defectos. El plano agéntico queda entre TRL 3 y 4: el ciclo gobernado funciona de punta a punta, pero su motor LLM aún no se ha medido con el arnés. El proyecto planificado tiene como meta llevar el conjunto a TRL 6, demostrado en un entorno relevante con un proveedor y una especificación reales.

5.6. Resultados de la planificación

5.6.1. Análisis de incertidumbre

Cada paquete se simula con una distribución beta-PERT sobre la red nivelada. La figura 16 muestra la distribución del plazo total en 10.000 iteraciones y la tabla siguiente resume los percentiles.

Tabla 31. *Resultados de la simulación de Monte Carlo*

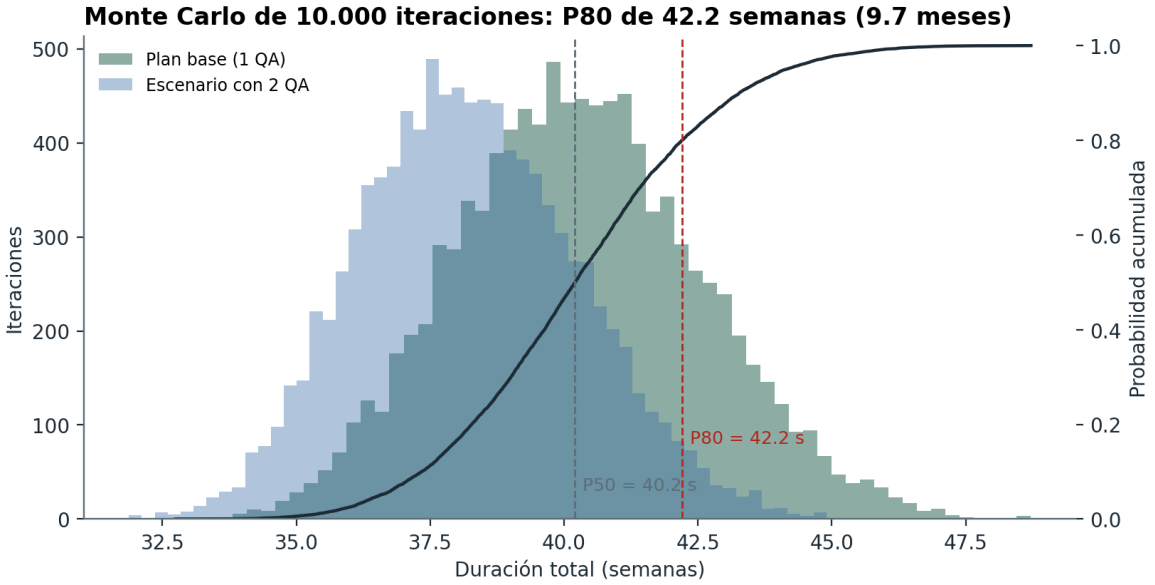
Indicador	P10	P50	P80	P90
Duración (semanas)	37,3	40,2	42,2	43,3
Duración (meses)	8,6	9,3	9,7	10,0
Costo (millones de COP)	947	1.013	1.059	1.085

Fuente: elaboración propia.

El plan comunica la fecha con su probabilidad: hay un 80 % de probabilidad de terminar en 42,2 semanas o menos (9,7 meses), y la probabilidad de cumplir el horizonte de 12 meses es prácticamente total. La diferencia entre P80 y P50 (2,0 semanas y COP 46 millones) es la reserva de contingencia.

Escenario con dos personas de QA. La nivelación revela que QA es el rol cuello de botella. Con una segunda persona de QA, el P50 baja de 40,2 a 38,1 semanas y el costo P50 baja de COP 1.013 millones a COP 982 millones, porque el ahorro en costos fijos y de dedicación continua supera el costo de la persona adicional. La recomendación del plan es incorporar una segunda persona de QA durante las fases F2 a F4.

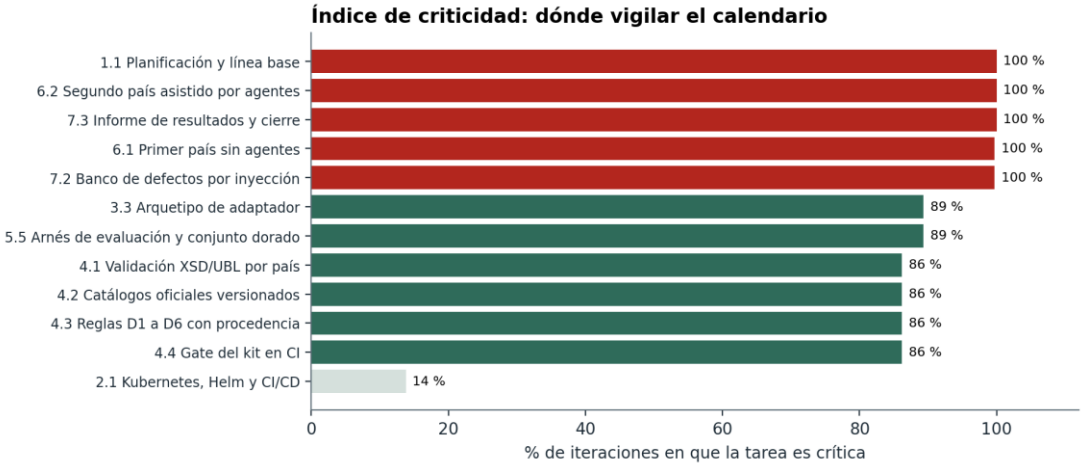
Figura 16. *Distribución del plazo total: plan base y escenario con dos personas de QA*



Fuente: elaboración propia. Salida del modelo de planificación (10.000 iteraciones, semilla fija).

El índice de criticidad, es decir, la proporción de iteraciones en que una tarea queda en la ruta crítica, indica dónde vigilar el calendario (figura 17).

Figura 17. *Índice de criticidad de los paquetes de trabajo*



Fuente: elaboración propia. Salida del modelo de planificación.

Un resultado relevante para la primera subpregunta: la gestión de accesos y credenciales (1.3) tiene un índice de criticidad de 0 % en el plan nivelado, gracias a que se inicia en la primera fase y en paralelo. Si se iniciara tarde, sería el camino crítico, y ningún agente podría acortarlo.

5.6.2. Punto de equilibrio

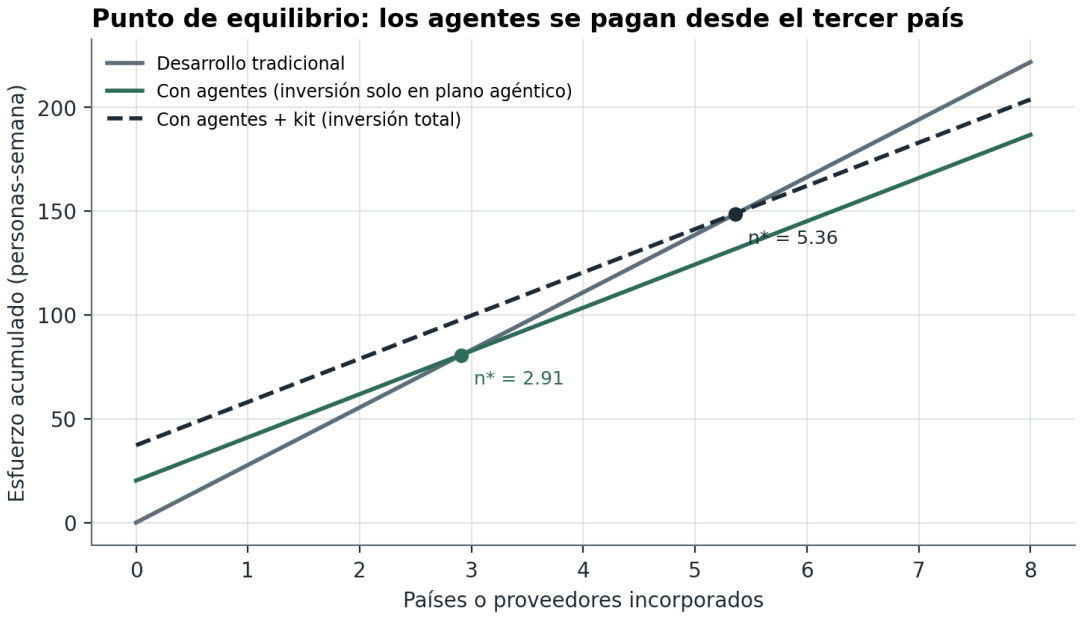
Si e_m es el esfuerzo de incorporar un país de forma manual, e_a el esfuerzo asistido y Eo la inversión inicial, la propuesta se paga a partir de:

$$n^* = Eo / (e_m - e_a)$$

Con el modelo, e_m = 27,7 p-s (paquete 6.1), e_a = 20,8 p-s (paquete 6.2) y el ahorro es de 6,9 p-s por país. Si Eo es solo la inversión en el plano agéntico (20,2 p-s), n = *2,91: los agentes se pagan desde el tercer país. Si se cuenta también el kit y el arquetipo (37,2 p-s), n* = 5,36. Esta segunda cifra es conservadora, porque el kit reduce rechazos aunque no haya agentes.

Para una organización que opera en cuatro o más países y cambia de proveedor con cierta frecuencia, la inversión se justifica por esfuerzo. Si opera en menos, debe justificarse por calidad y trazabilidad, y el plan lo declara así.

Figura 18. *Punto de equilibrio en esfuerzo acumulado*



Fuente: elaboración propia. Salida del modelo de planificación. e_a es una hipótesis que el proyecto mide en la fase F5.

5.7. Amenazas a la validez

- **Diseño por el mismo autor:** la especificación, las reglas y los mutantes los diseñó la misma persona, lo que puede favorecer la detección. En el proyecto, los mutantes los propone un tercero.
- **Fixtures por construcción:** la detección del 100 % en CI depende de que los fixtures cubran los casos; su cobertura es en sí un requisito de calidad.
- **Tamaño del banco:** 22 mutantes son suficientes para mostrar el patrón, no para estimar tasas con precisión estadística.
- **Motor heurístico:** la exactitud medida es la de la línea base, no la de un LLM.
- **Sin proveedor real:** la PoC no mide rechazos externos ni tiempos de proveedor.

5.8. Cumplimiento de los objetivos

La tabla siguiente contrasta los resultados con cada uno de los objetivos planteados en el apartado 1.2. Los tres objetivos específicos se cumplen con la evidencia indicada, y el objetivo general se cumple con la salvedad de que el prototipo trabaja sobre una especificación y datos sintéticos.

Tabla 32. *Cumplimiento de los objetivos específicos y del objetivo general*

Objetivo	Evidencia	Estado
OE1. Caracterizar los defectos de mapeo con una taxonomía de al menos seis tipos	Taxonomía D1 a D6, cada tipo con su regla de verificación y su operador de mutación (apartado 2.2 y banco de 22 mutantes)	Cumplido
OE2. Diseñar la arquitectura y el kit de verificación con al menos diez decisiones y una detección mínima del 80 % sin falsos positivos	Diez decisiones arquitectónicas documentadas (apartado 4.5 y anexo A); detección del 86 % en ejecución y del 100 % con integración continua, con cero falsos positivos (apartado 5.2)	Cumplido en laboratorio (TRL 4)
OE3. Elaborar el plan de ejecución con plazo y costo al 80 % de confianza mediante Monte Carlo	EDT de 24 paquetes, cronograma nivelado, presupuesto, registro de riesgos y pipeline de integración y despliegue continuos (apartados 4.6 a 4.10); plazo P80 de 42,2 semanas y costo P80 de COP 1.059 millones (apartado 5.6)	Cumplido
Objetivo general. Planificar la arquitectura validando su viabilidad técnica con un prototipo evaluado con un banco de defectos	Prototipo funcional con 32 pruebas automatizadas y banco de 22 defectos inyectados; plan cuantificado de punto a punto	Cumplido, con las limitaciones del apartado 5.7

Fuente: elaboración propia.

6. Conclusiones

6.1. Síntesis del trabajo

Este trabajo partió de un problema concreto y recurrente: cada vez que una organización que factura en varios países incorpora un nuevo país o un nuevo proveedor tecnológico, debe construir a mano el mapeo de aproximadamente cien campos entre su modelo canónico y el formato nacional, y los defectos de ese mapeo solo se descubren cuando el proveedor o la autoridad tributaria rechazan el documento. El análisis del problema mostró que la causa no es única: confluyen un método manual, conocimiento tácito concentrado en pocas personas, normas que cambian por versión y una única barrera previa al envío, la validación de esquema, que no detecta los errores semánticos.

La solución planificada combina dos ideas. La primera es acelerar la propuesta de mapeos con agentes de inteligencia artificial. La segunda, y la más importante, es no confiar en ellos: un kit de verificación determinista, trazable hasta la norma, decide qué propuesta puede llegar a revisión humana, y nada se despliega sin la aprobación de una persona. Sobre esa arquitectura se elaboró el plan de ejecución, con su equipo, cronograma, presupuesto y riesgos, y se construyó un prototipo para comprobar con datos que el diseño funciona.

6.2. Respuesta a la pregunta guía

La arquitectura agéntica con verificación determinista reduce el esfuerzo de incorporar un país en 6,9 personas-semana según el modelo de planificación, y la evidencia de la prueba de concepto indica que puede hacerlo sin degradar la validez: el kit detectó el 86 % de los defectos en ejecución y el 100 % con CI, sin falsos positivos.

- **Subpregunta 1:** una parte del esfuerzo no es automatizable. Los accesos y credenciales solo dejan de ser críticos si se gestionan desde el inicio y en paralelo, y el plan los trata con gestión, no con IA.
- **Subpregunta 2:** el esquema solo detecta el 23 %; las reglas con procedencia, el 86 %; los fixtures cierran el resto. El agente heurístico acertó el 95,8 % de los campos, y su único error lo bloqueó el kit antes de la revisión humana.
- **Subpregunta 3:** los agentes se pagan desde el país número 2,9, y la inversión total, desde el país número 5,4.

Estas tres respuestas se refuerzan entre sí. La primera delimita lo que la inteligencia artificial no resuelve, la segunda muestra que puede usarse sin comprometer la validez de lo emitido y la tercera indica a partir de cuántos países la inversión se justifica. Tomadas juntas, sostienen que la propuesta es técnicamente viable y económicamente razonable para una

organización que prevé incorporar varios países, y la evidencia sugiere que su valor depende menos de la capacidad del modelo de lenguaje que de la calidad del verificador que lo rodea.

6.3. Discusión de los objetivos

El primer objetivo específico, caracterizar los defectos de mapeo, se cumplió con la taxonomía de seis tipos D1 a D6. Su valor no está solo en la clasificación: cada tipo quedó asociado a una regla de verificación y a un operador de mutación, de modo que la misma taxonomía sirvió para diagnosticar el problema, para construir el verificador y para evaluarlo. El análisis de Pareto, basado en juicio experto, mostró además que tres tipos (código de catálogo incorrecto, campo exigido ausente y campos intercambiados) concentran el 80,8 % de los defectos estimados, lo que orientó dónde invertir más reglas y más casos de prueba.

El segundo objetivo, diseñar la arquitectura y el kit de verificación, se cumplió con diez decisiones arquitectónicas documentadas y con un prototipo que superó la meta de detección. En el banco de 22 defectos inyectados, la validación de esquema detectó el 23 %, el kit en ejecución el 86 % y el kit con ejemplos validados en integración continua el 100 %, sin ningún falso positivo. Los tres defectos que escapan en ejecución no son un fallo del diseño, sino la prueba de que el control debe ser doble: hay errores que ninguna regla puede ver sin conocimiento del negocio y que solo aparecen al comparar con ejemplos validados por personas.

El tercer objetivo, elaborar el plan de ejecución, se cumplió con una estructura de desglose de 24 paquetes de trabajo, un cronograma nivelado por recursos, un presupuesto asignado a tareas y perfiles y un registro de riesgos con sus respuestas. La simulación de Monte Carlo permitió expresar el plazo y el costo con su probabilidad: 42,2 semanas y COP 1.059 millones con una confianza del 80 %. La nivelación fue determinante, porque un cronograma calculado solo con dependencias técnicas sobreasignaba al ingeniero de IA, al de operaciones y a QA, y por eso no era ejecutable.

Los tres objetivos dependen unos de otros: sin taxonomía no hay reglas, sin reglas no hay kit que medir y sin un kit medido no hay base para estimar cuánto esfuerzo ahorran los agentes. Por eso se cumplen como una cadena y no como piezas independientes.

6.4. Contribuciones del trabajo

El trabajo hace cinco contribuciones, las mismas que se anunciaron como elementos innovadores en el apartado 1.3. La primera es una taxonomía de seis tipos de defecto de mapeo, cada uno con su regla de verificación y su operador de mutación. La segunda es un kit de verificación determinista cuyas reglas citan la sección y la huella digital del documento normativo del que provienen. La tercera es un ciclo gobernado en el que los agentes solo

proponen mediante solicitudes de cambio y nada llega a producción sin superar el kit y la aprobación de una persona. La cuarta es un banco de defectos por inyección que mide con datos la eficacia de cada mecanismo de verificación. La quinta es un plan de ejecución cuantitativo, con plazo y costo expresados con su probabilidad, un presupuesto calibrado con referentes del mercado y un punto de equilibrio que indica cuándo se recupera la inversión.

6.5. Implicaciones para la práctica

Para una organización que factura en varios países, el resultado más útil del trabajo no es una herramienta, sino un criterio de diseño: tratar la salida de un modelo de lenguaje como una propuesta sujeta a verificación objetiva, y no como una decisión. Ese criterio es aplicable más allá de la facturación electrónica a cualquier dominio regulado en el que un error tenga consecuencias legales o fiscales.

De él se desprenden tres implicaciones. La primera es que el orden de las fases importa: el kit llega antes que los agentes porque aporta valor por sí solo y es la condición para usarlos con seguridad. La segunda es que el conocimiento que hoy es tácito, como los códigos acordados de palabra o las reglas escondidas en código heredado, se convierte en reglas escritas, versionadas y trazables a la norma, lo que reduce la dependencia de pocas personas. La tercera es que la inteligencia artificial cambia el papel de los cargos en lugar de eliminarlos: el desarrollador de integración pasa de transcribir especificaciones a revisar propuestas con evidencia, y el especialista tributario, de validar documentos uno a uno a mantener reglas.

Finalmente, el modelo de planificación muestra que la inversión no se justifica con un solo país. Con los agentes el punto de equilibrio está en 2,9 países y con la inversión completa en 5,4, de modo que la propuesta conviene a organizaciones con una expansión sostenida y no a quien integrará un único país.

6.6. Limitaciones

Las principales limitaciones son el uso de una especificación sintética, la ausencia de medición con un LLM real, la estimación del esfuerzo asistido como hipótesis y las tarifas supuestas. Todas están declaradas, y todas tienen una actividad del plan que las resuelve.

La más importante es que el prototipo trabaja sobre una especificación y datos sintéticos, diseñados por el mismo autor que las reglas y los mutantes. Esto significa que el 86 % y el 100 % de detección demuestran que el diseño funciona en condiciones controladas, pero no permiten estimar la tasa que se obtendría con documentos y proveedores reales: el banco de 22 defectos basta para mostrar el patrón, no para estimar tasas con precisión. La

detección del 100 % con integración continua, además, depende de que los ejemplos validados cubran los casos, y su cobertura se construyó con el mismo criterio.

La segunda limitación es que el motor de lenguaje del agente de mapeo no se midió con un modelo real: la exactitud del 95,8 % corresponde al motor heurístico, que sirve de línea base, y el motor basado en un modelo de lenguaje se probó con un cliente simulado. La tercera es que el ahorro de 6,9 personas-semana por país es una hipótesis del modelo de planificación, que la fase F5 del proyecto comprueba con la primera incorporación real. La cuarta es que las tarifas de tres roles no tienen una fuente pública verificada; el análisis de sensibilidad muestra que, con tarifas un 20 % menores o un 25 % mayores, el plazo no cambia y el costo en el percentil 50 se mueve entre COP 855 y 1.211 millones, pero la cifra exacta sigue siendo una estimación. Estas amenazas se detallan en el apartado 5.7.

6.7. Líneas de trabajo futuro

- Medir el motor LLM con el arnés de evaluación, comparando modelos y su costo por propuesta.
- Ampliar el banco con mutantes propuestos por terceros y con defectos reales anonimizados.
- Validar la matriz de decisión y las estimaciones con un panel de expertos.
- Generar automáticamente fixtures adversarios a partir de la especificación.
- Estudiar un modelo semántico común para Latinoamérica, inspirado en EN 16931, como capa del canónico.

Estas líneas se ordenan por el riesgo que reducen. La primera y la tercera atacan directamente las limitaciones principales: medir el motor de lenguaje con el arnés de evaluación y validar la matriz de decisión y las estimaciones con un panel de expertos darían a las cifras del trabajo una base empírica externa. La segunda y la cuarta ampliarían la cobertura del banco de defectos, que es la evidencia central de la propuesta. La quinta, el estudio de un modelo semántico común para Latinoamérica, es de mayor alcance y podría reducir de raíz el problema que originó este trabajo: la repetición del mapeo en cada país.

La contribución central del trabajo es mostrar que la IA generativa puede entrar en un dominio regulado si se gobierna como cualquier otra fuente de cambio: con un verificador objetivo, trazabilidad hasta la norma, decisión humana y un plan que mide su costo y su riesgo.

En síntesis, el trabajo muestra que es posible acelerar una tarea repetitiva y propensa a errores con inteligencia artificial sin renunciar a las garantías que exige un documento tributario, siempre que la velocidad de la propuesta se equilibre con un control

independiente del modelo. La planificación presentada convierte esa idea en un proyecto ejecutable, con equipo, plazo, costo y riesgos cuantificados, y el prototipo aporta la primera evidencia de que el diseño se comporta como se espera. Lo que sigue es llevarlo a un entorno real, con un proveedor y una especificación reales, para comprobar en la práctica lo que aquí se demostró en laboratorio.

Referencias Bibliográficas

Banco Interamericano de Desarrollo [BID]. (s. f.). *Usos prácticos de la factura electrónica para el control tributario: experiencias de las administraciones tributarias*. Blog Gestión Fiscal. https://blogs.iadb.org/gestion-fiscal/es/usos-practicos-de-la-factura-electronica-para-el-control-tributario-experiencias-de-las-administraciones-tributarias

Barreix, A., y Zambrano, R. (Eds.). (2018). *Factura electrónica en América Latina* (Monografía del BID N.º 595). Banco Interamericano de Desarrollo; Centro Interamericano de Administraciones Tributarias. https://publications.iadb.org/es/factura-electronica-en-america-latina

Barron, F. H., y Barrett, B. E. (1996). Decision quality using ranked attribute weights. *Management Science, 42*(11), 1515-1523. https://doi.org/10.1287/mnsc.42.11.1515

Basili, V. R., Caldiera, G., y Rombach, H. D. (1994). The goal question metric approach. En J. J. Marciniak (Ed.), *Encyclopedia of software engineering* (pp. 528-532). Wiley.

Bass, L., Clements, P., y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Brown, S. (s. f.). *The C4 model for visualising software architecture*. https://c4model.com

Buk. (2026). *Costo real de un empleado en Colombia 2026*. https://www.buk.co/blog/costo-real-de-un-empleado-en-colombia-2026

Cabrera, L., Romero, S., y Aguirre, J. (2026). De la factura física al documento digital: transformación tributaria y deducción del gasto en Perú. *Revista InveCom, 6*(1), 1-10. https://zenodo.org/records/15529473

Calijuri, M. (2024). Revolucionando la tributación: la administración tributaria 3.0 ya está aquí (Nota técnica). Banco Interamericano de Desarrollo. https://publications.iadb.org/publications/spanish/document/revolucionando-la-tributacion-la-administracion-tributaria-ya-esta-aqui.pdf

CMMI Product Team. (2010). *CMMI for Development, version 1.3* (CMU/SEI-2010-TR-033). Software Engineering Institute, Carnegie Mellon University.

Colombia Move. (2026). *Salarios en Colombia por industria: rangos reales 2026* [Entrada de blog]. https://colombiamove.com/blog/es/average-salaries-colombia-by-industry/

Comité Europeo de Normalización [CEN]. (2017). *EN 16931-1:2017. Electronic invoicing – Part 1: Semantic data model of the core elements of an electronic invoice*.

De Velazco, J. L. (2016). La facturación electrónica en el Perú. *Lidera, 11*, 4-10. https://revistas.pucp.edu.pe/index.php/revistalidera/article/view/16873

Departamento Nacional de Planeación [DNP]. (2025). *Documento CONPES 4144: Política Nacional de Inteligencia Artificial*. Consejo Nacional de Política Económica y Social.

Dirección de Impuestos y Aduanas Nacionales [DIAN]. (2023). *Resolución 000165 de 2023, por la cual se desarrolla el sistema de facturación, los proveedores tecnológicos, se adopta la versión 1.9 del anexo técnico de factura electrónica de venta y se dictan otras disposiciones*.

Dirección de Impuestos y Aduanas Nacionales [DIAN]. (2024). *Resolución 000008 de 2024, por la cual se modifica el artículo 23 y parcialmente el parágrafo del artículo 62 de la Resolución 000165 de 2023*. https://normograma.mintic.gov.co/docs/pdf/resolucion_dian_0008_2024.pdf

Escobar, M. (2019). La factura electrónica. Un caso de análisis del principio de equivalencia funcional para el derecho chileno. *Revista de Derecho, 242*, 159-188. https://revistas.udec.cl/index.php/revista_de_derecho/article/view/752

Giraldo-Hurtado, J. S. (2023). *Catálogo de herramientas de gestión de la configuración que implementan la metodología GitOps* [Trabajo fin de máster, Universidad Internacional de La Rioja]. Re-UNIR. https://reunir.unir.net/handle/123456789/15912

Gray, C. F., y Larson, E. W. (2009). *Administración de proyectos* (4.ª ed.). McGraw-Hill.

Hernández Sampieri, R., Fernández Collado, C., y Baptista Lucio, P. (2014). *Metodología de la investigación* (6.ª ed.). McGraw-Hill.

Hevner, A. R., March, S. T., Park, J., y Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75-105. https://doi.org/10.2307/25148625

Hohpe, G., y Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., y Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology, 33*(8).

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., y Liu, T. (2025). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems, 43*(2). https://doi.org/10.1145/3703155

Ishikawa, K. (1986). *¿Qué es el control total de calidad? La modalidad japonesa*. Editorial Norma.

ISO/IEC. (2023). *ISO/IEC 25010:2023. Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model.*

Jia, Y., y Harman, M. (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering, 37*(5), 649-678.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., y Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? En *International Conference on Learning Representations (ICLR 2024)*. https://arxiv.org/abs/2310.06770

Juran, J. M. (1990). *Juran y la planificación para la calidad*. Díaz de Santos.

Kendall, K. E., y Kendall, J. E. (2011). *Análisis y diseño de sistemas* (8.ª ed.). Pearson Educación.

Kolisch, R. (1996). Serial and parallel resource-constrained project scheduling methods revisited: Theory and computation. *European Journal of Operational Research, 90*(2), 320-333.

La FM. (2024). *Las profesiones mejor pagadas: se gana hasta $26 millones al mes*. https://www.lafm.com.co/empleo/las-profesiones-mejor-pagadas-se-gana-hasta-26-millones-al-mes

Leguizamón, J. D., y Lugo, S. (2024). Implementación de la facturación electrónica en América Latina: revisión de literatura. *Arandu UTIC, 11*(2), 2564-2587. https://doi.org/10.69639/arandu.v11i2.446

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., y Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459-9474.

Liu, Y., Peña, E., Santos, A., Wu, E., y Freire, J. (2025). Magneto: Combining small and large language models for schema matching. *Proceedings of the VLDB Endowment, 18*(8), 2681-2694. https://doi.org/10.14778/3742728.3742757

Malcolm, D. G., Roseboom, J. H., Clark, C. E., y Fazar, W. (1959). Application of a technique for research and development program evaluation. *Operations Research, 7*(5), 646-669.

Model Context Protocol. (2026). *Model Context Protocol specification* (revisión 2026-07-28). https://github.com/modelcontextprotocol/specification/releases/tag/2026-07-28

Narayan, A., Chami, I., Orr, L., y Ré, C. (2022). Can foundation models wrangle your data? *Proceedings of the VLDB Endowment, 16*(4), 738-746. https://www.vldb.org/pvldb/vol16/p738-narayan.pdf

National Institute of Standards and Technology [NIST]. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). https://doi.org/10.6028/NIST.AI.100-1

Nygard, M. (2011). *Documenting architecture decisions*. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

OASIS. (2013). *Universal Business Language version 2.1* [OASIS Standard]. http://docs.oasis-open.org/ubl/os-UBL-2.1/UBL-2.1.html

OCDE y Eurostat. (2006). *Manual de Oslo: guía para la recogida e interpretación de datos sobre innovación* (3.ª ed.). Grupo Tragsa.

OCDE. (2022). *Tax administration 3.0 and electronic invoicing: Initial findings*. https://www.oecd.org/en/publications/tax-administration-3-0-and-electronic-invoicing_2ffc88ed-en.html

Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hróbjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., McDonald, S., … Alonso-Fernández, S. (2021). Declaración PRISMA 2020: una guía actualizada para la publicación de revisiones sistemáticas. *Revista Española de Cardiología, 74*(9), 790-799. https://doi.org/10.1016/j.recesp.2021.06.016

Parciak, M., Vandevoort, B., Neven, F., Peeters, L. M., y Vansummeren, S. (2024). Schema matching with large language models: An experimental study. En *Proceedings of Workshops at the 50th International Conference on Very Large Data Bases (VLDB 2024)*. https://vldb.org/workshops/2024/proceedings/TaDA/TaDA.8.pdf

Peffers, K., Tuunanen, T., Rothenberger, M. A., y Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45-77. https://doi.org/10.2753/MIS0742-1222240302

Portafolio. (2026). *Salarios en tecnología en Colombia 2026: cuánto ganan perfiles en inteligencia artificial, datos y software*.

https://www.portafolio.co/tecnologia/salarios-en-tecnologia-en-colombia-2026-cuanto-ganan-perfiles-en-inteligencia-artificial-datos-y-software-491946

Pressman, R. S. (2010). *Ingeniería del software: un enfoque práctico* (7.ª ed.). McGraw-Hill.

Project Management Institute [PMI]. (2021). *Guía de los fundamentos para la dirección de proyectos (Guía del PMBOK)* (7.ª ed.).

Quevedo-Tumailli, V., Arias, S. E., Ortega, V. A., y Ortega-Tenezaca, B. (2025). Impacto de modelos de lenguaje de gran tamaño en calidad y eficiencia de generación de código: revisión sistemática de literatura. *Novasinergia, 8*(1), 52-66. https://doi.org/10.37135/ns.01.15.10

Rahm, E., y Bernstein, P. A. (2001). A survey of approaches to automatic schema matching. *The VLDB Journal, 10*(4), 334-350.

Ramírez-Álvarez, J., Oliva, N., y Andino, M. (2022). Cumplimiento tributario y facturación electrónica en Ecuador: evaluación de impacto. *Problemas del Desarrollo. Revista Latinoamericana de Economía, 53*(208), 97-123. https://doi.org/10.22201/iiec.20078951e.2022.208.69712

Romero, C. (1993). *Teoría de la decisión multicriterio: conceptos, técnicas y aplicaciones*. Alianza Editorial.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

Superintendencia Nacional de Aduanas y de Administración Tributaria [SUNAT]. (2010). *Resolución de Superintendencia N.º 188-2010/SUNAT*.

Valora Analitik. (2026). *Estos trabajos en Colombia están pagando hasta $23 millones mensuales*. https://www.valoraanalitik.com/?p=572216

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., y Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science, 18*(6), 186345. https://doi.org/10.1007/s11704-024-40231-1

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., y Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. En *International Conference on Learning Representations (ICLR 2023)*. https://arxiv.org/abs/2210.03629

Anexos

Anexo A. Decisiones arquitectónicas completas

ADR-01. Topics de Kafka por país y proveedor

Estado: aceptada. **Contexto:** Cada país tiene volúmenes, ventanas de cierre y proveedores distintos. Un fallo del proveedor de un país no puede detener la emisión de los demás.

- **Cola única con filtrado por cabecera:** Simple de operar; un consumidor lento o un mensaje envenenado afecta a todos los países.
- **Topic por país y proveedor (elegida):** Aislamiento de fallos, escalado y retención por país; más topics que gobernar.
- **Colas por tipo de documento:** Aísla por tipo, no por país; no resuelve el aislamiento de proveedores.

Decisión: Se usan topics con la convención invoice.<país>.<proveedor>.<evento>, con particionado por número de documento para conservar el orden por comprobante.

Consecuencias: Positivas: un incidente queda contenido en su país; la retención y las réplicas se ajustan por volumen. Negativas: más topics y ACL que administrar, mitigado con plantillas de infraestructura como código.

Escenario de calidad: Fiabilidad: un proveedor caído durante 2 horas no degrada la emisión de los otros países (0 mensajes perdidos, latencia del resto sin cambio).
Evidencia en la PoC: Fuera del alcance de la PoC (no incluye Kafka); se valida en la fase F1 del proyecto planificado.

ADR-02. Adaptador por proveedor generado con arquetipo

Estado: aceptada. **Contexto:** Los formatos de los proveedores son heterogéneos (JSON, texto por segmentos, XML/UBL). Un transformador genérico configurable tiende a convertirse en un motor de reglas opaco.

- **Transformador genérico configurable:** Un solo componente; la configuración crece hasta ser un lenguaje propio difícil de probar.
- **Adaptador por proveedor desde arquetipo (elegida):** Código explícito y probable por proveedor; el arquetipo fija estructura, pruebas y verificación.
- **iPaaS con mapeo visual:** Rápido al inicio; dependencia del fabricante y mapeo manual en cada país.

Decisión: Cada proveedor tiene su adaptador generado con un arquetipo Maven que incluye el kit de verificación embebido y las pruebas de contrato.

Consecuencias: Positivas: cada adaptador es pequeño, probable y desplegable por separado. Negativas: duplicación controlada entre adaptadores, que se reduce moviendo lo común al kit y a librerías compartidas.

Escenario de calidad: Mantenibilidad: incorporar un proveedor nuevo parte de un esqueleto que ya compila, pasa el kit y tiene su pipeline. **Evidencia en la PoC:** El transformador de la PoC separa el mapeo (datos) del motor de renderizado: el arquetipo empaqueta ambos.

ADR-03. Python para el plano agéntico

Estado: aceptada. **Contexto:** El plano de integración es Java. Los agentes necesitan el ecosistema de LLM, MCP y orquestación de agentes, donde Python es la referencia.

- **Spring AI en Java:** Un solo lenguaje; ecosistema de agentes y MCP menos maduro.
- **Python con LangGraph y SDK de MCP (elegida):** Ecosistema más maduro y documentado; dos lenguajes en el equipo.
- **TypeScript:** Buen SDK de MCP; menor afinidad con el equipo de datos e IA.

Decisión: El plano agéntico se implementa en Python 3.12. La frontera con Java es el kit de verificación, expuesto por HTTP y por CLI.

Consecuencias: Positivas: acceso directo al ecosistema de IA. Negativas: dos cadenas de herramientas; el contrato del kit debe ser estable y versionado.

Escenario de calidad: Mantenibilidad: el plano agéntico puede cambiar de proveedor de LLM sin tocar el plano de integración. **Evidencia en la PoC:** La PoC implementa el agente, el servidor MCP y la API en Python, e invoca el kit Java por CLI y HTTP.

ADR-04. Los agentes solo proponen mediante pull request

Estado: aceptada. **Contexto:** Un LLM puede producir salidas plausibles e incorrectas. En facturación, una salida incorrecta es un documento rechazado o una contingencia fiscal.

- **Agentes con permiso de despliegue:** Máxima velocidad; responsabilidad difusa y riesgo inaceptable en un dominio regulado.
- **Agentes que proponen por pull request (elegida):** Toda salida pasa por revisión y por el pipeline; añade latencia humana.

- **Sin agentes:** Elimina el riesgo de IA; no reduce el costo de incorporar países.

Decisión: Los agentes solo crean ramas y pull requests. No tienen permiso de merge ni acceso a producción. Cada propuesta queda registrada en agent_proposal con su verificación, revisor y decisión.

Consecuencias: Positivas: responsabilidad clara y trazabilidad completa. Negativas: la revisión humana es un cuello de botella, mitigado porque el kit filtra antes lo que no puede aprobarse.

Escenario de calidad: Seguridad (responsabilidad): cualquier cambio en producción se puede rastrear hasta su propuesta, su verificación y la persona que la aprobó. **Evidencia en la PoC:** La API de revisión impide aprobar una propuesta bloqueada por el kit (HTTP 409), comprobado en las pruebas automatizadas.

ADR-05. Verificación determinista como gate en ejecución y en CI

Estado: aceptada. **Contexto:** La revisión humana por sí sola no detecta de forma fiable códigos válidos pero equivocados ni intercambios de campos. Se necesita un verificador que no dependa del juicio del modelo.

- **Revisión humana como único control:** Sin inversión inicial; detección irregular y no medible.
- **Solo validación XSD:** Barata; en la PoC detecta el 23 % de los mutantes.
- **Kit en ejecución + fixtures en CI (elegida):** En la PoC detecta el 86 % en ejecución y el 100 % con CI, sin falsos positivos.

Decisión: El kit se embebe en cada adaptador (verificación en ejecución) y se ejecuta como gate obligatorio en el pipeline con fixtures validados por personas.

Consecuencias: Positivas: control objetivo, medible y trazable a la norma. Negativas: hay que mantener reglas y fixtures por versión normativa; se asume como parte del costo de cada país.

Escenario de calidad: Adecuación funcional: el 100 % de los comprobantes pasa XSD y reglas antes de salir; ninguna propuesta que falle el kit puede aprobarse. **Evidencia en la PoC:** Banco de 22 mutantes: esquema 23 %, kit en ejecución 86 %, kit + CI 100 %; 0 falsos positivos en 1.224 verificaciones correctas.

ADR-06. Respuesta con fuente primaria y respaldo

Estado: aceptada. **Contexto:** La respuesta del proveedor puede perderse o llegar incompleta. La constancia oficial de la autoridad también es recuperable.

- **Solo la respuesta del proveedor:** Simple; un fallo del proveedor deja comprobantes sin estado.
- **Fuente primaria y respaldo (elegida):** Más resiliente; requiere correlación y reconciliación.
- **Consulta directa a la autoridad:** No siempre disponible y depende de cada país.

Decisión: La respuesta del proveedor es la fuente primaria; la constancia oficial en UBL 2.1 es el respaldo. Ambas se correlacionan por correlationId y número de documento.

Consecuencias: Positivas: menos comprobantes en estado incierto. Negativas: lógica de reconciliación y pruebas adicionales.

Escenario de calidad: Fiabilidad: ningún comprobante autorizado queda sin estado final tras la reconciliación diaria. **Evidencia en la PoC:** Fuera del alcance de la PoC; se valida en la fase F3.

ADR-07. Archivo legal asíncrono de mejor esfuerzo con reintento

Estado: aceptada. **Contexto:** El archivo legal (PDF, XML y constancia) es obligatorio, pero no debe bloquear la actualización del estado del comprobante.

- **Archivo síncrono bloqueante:** Consistencia inmediata; un fallo del almacenamiento detiene la emisión.
- **Archivo asíncrono con reintento y alerta (elegida):** Disponibilidad del flujo principal; consistencia eventual controlada.

Decisión: El archivado ocurre de forma asíncrona, con reintentos, cuarentena en DLQ y alerta si se supera el umbral. El almacenamiento es inmutable (WORM) con retención por país.

Consecuencias: Positivas: el flujo principal no depende del almacenamiento. Negativas: ventana corta de consistencia eventual que debe monitorearse.

Escenario de calidad: Disponibilidad: una caída del almacenamiento de 30 minutos no detiene la emisión y se recupera sin pérdida. **Evidencia en la PoC:** Fuera del alcance de la PoC; se valida en la fase F3.

ADR-08. Schema Registry con compatibilidad BACKWARD

Estado: aceptada. **Contexto:** El modelo canónico evolucionará (por ejemplo, el correo del adquirente pasó a ser obligatorio). Los consumidores no se despliegan todos a la vez.

- **Contratos sin registro:** Libertad total; roturas silenciosas entre servicios.

- **Registro con compatibilidad BACKWARD (elegida):** Evolución segura; disciplina en los cambios de esquema.

Decisión: El modelo canónico InvoiceRequest se registra en Schema Registry con compatibilidad BACKWARD; los campos nuevos nacen opcionales con valor por defecto.

Consecuencias: Positivas: los consumidores antiguos siguen funcionando. Negativas: un campo obligatorio requiere un proceso de dos versiones.

Escenario de calidad: Mantenibilidad: un cambio del canónico no rompe a ningún consumidor desplegado. **Evidencia en la PoC:** El defecto D4 de la PoC reproduce justamente un campo obligatorio que el canónico antiguo no traía.

ADR-09. Migraciones de base de datos con Flyway

Estado: aceptada. **Contexto:** Cada adaptador persiste estado y auditoría. Los entornos SIT, UAT y PROD deben tener el mismo esquema.

- **Scripts manuales:** Sin herramienta; deriva entre entornos y errores de orden.
- **Flyway versionado en el repositorio (elegida):** Reproducible y auditable; disciplina de versiones.

Decisión: Cada servicio versiona sus migraciones con Flyway; el pipeline las aplica antes del despliegue.

Consecuencias: Positivas: entornos reproducibles. Negativas: las migraciones destructivas exigen un procedimiento de dos pasos.

Escenario de calidad: Mantenibilidad: cualquier entorno se reconstruye desde cero con el mismo esquema. **Evidencia en la PoC:** La PoC usa SQLite con esquema declarado en código para agent_proposal; en el proyecto se migra con Flyway.

ADR-10. RAG con procedencia para leer especificaciones

Estado: aceptada. **Contexto:** Las especificaciones son largas y cambian por versión. El agente necesita citar exactamente de dónde sale cada decisión.

- **Especificación completa en el prompt:** Simple; costoso, sin trazas y sin control de versión.
- **RAG con procedencia (elegida):** Cada fragmento lleva sección y hash del documento; costo menor por consulta.
- **Ajuste fino del modelo:** Costoso y rígido ante cambios normativos.

Decisión: El lector de especificaciones devuelve fragmentos con su sección y el SHA-256 del documento. Toda regla y toda propuesta guardan esa procedencia.

Consecuencias: Positivas: trazabilidad regla → norma y auditoría de cambios normativos. Negativas: hay que reindexar cada versión de la norma.

Escenario de calidad: Seguridad (responsabilidad): cada regla y propuesta se rastrea hasta la sección y la versión exacta del documento que la origina. **Evidencia en la PoC:** El servidor MCP devuelve sección y huella; cada hallazgo del kit incluye fuente, sección, hash y versión de la regla.

Anexo B. Supuestos de planificación

Los supuestos son del autor. Los costos por rol se calibraron contra los referentes públicos del apartado 4.8; los de IA/ML, QA y especialista tributario no tienen una fuente pública verificada y se declaran como supuestos.

Tabla 33. *Costo empresa mensual por rol (supuesto)*

Rol	COP por mes (tiempo completo)
Gerente del proyecto	18.000.000
Arquitecto de soluciones	24.000.000
Líder técnico de integración	19.000.000
Desarrolladores backend Java (3)	13.000.000
Ingeniero de IA/ML	18.000.000
Ingeniero DevOps/SRE	18.000.000
QA de automatización	10.000.000
Especialista tributario	14.000.000

Fuente: elaboración propia.

Tabla 34. *Otros supuestos del modelo*

Supuesto	Valor
Semanas por mes	4,33
TRM	4.000 COP/USD
Nube (tres entornos)	18.000.000 COP/mes
Licencias (única vez)	25.000.000 COP
Herramientas	3.000.000 COP/mes
Precio LLM	USD 3,00 por millón de tokens de entrada y USD 15,00 de salida
Iteraciones de Monte Carlo y semilla	10.000; semilla 2026
Distribución de duraciones	Beta-PERT ($\lambda = 4$) sobre (o, m, p)
Nivelación	Esquema serial con prioridad por inicio tardío; capacidad por rol según la tabla de equipo

Fuente: elaboración propia.

Anexo C. Resultados detallados del banco de defectos

Tabla 35. *Resultado por mutante*

Mutante	Descripción	Docs. afectados	Esquema	Ejecución	CI	Detección por doc.
D1-a	Factura codificada como nota débito (92), código válido pero equivocado	214	no	sí	sí	100 %
D1-b	Notas crédito codificadas como factura (01)	56	no	sí	sí	100 %
D1-c	Código de tipo de documento inexistente (07) tomado de una nota verbal	214	no	sí	sí	100 %
D1-d	Documento nacional (CC) codificado como empresa (A1)	107	no	sí	sí	100 %
D1-e	Pasaporte codificado como documento de extranjero (A3); formatos idénticos	35	no	no	sí	0 %
D1-f	IVA general (IVA19) codificado como IVA reducido (T02)	272	no	sí	sí	100 %
D1-g	Beta: nota débito con código inexistente (93)	30	no	sí	sí	100 %
D2-a	Cantidad y precio unitario intercambiados (producto idéntico)	300	no	no	sí	0 %
D2-b	Precio unitario y valor de línea intercambiados	295	no	sí	sí	100 %
D2-c	Total a pagar tomado del subtotal	290	no	sí	sí	100 %
D2-d	Impuesto total tomado del subtotal	300	no	sí	sí	100 %
D3-a	Identificador de perfil en minúsculas	300	no	sí	sí	100 %
D3-b	Nombre del esquema tributario con espacio final	272	no	sí	sí	100 %
D3-c	Nombre del esquema tributario en mayúscula inicial	290	no	sí	sí	100 %
D3-d	Moneda en minúsculas	300	no	sí	sí	100 %
D4-a	Correo del adquirente sin mapear (el canónico antiguo no lo traía)	300	sí	sí	sí	100 %
D4-b	Correo del adquirente mapeado a un literal vacío	300	no	sí	sí	100 %
D4-c	Fecha de emisión sin mapear	300	sí	sí	sí	100 %
D5-a	Se envía LegacyOrderReference, retirado en v2	300	sí	sí	sí	100 %
D5-b	Se envía TaxExchangeRateLegacy, retirado en v2	300	sí	sí	sí	100 %
D6-a	Descripción sin sanear el separador reservado	42	sí	sí	sí	100 %
D6-b	Nombre del adquirente sin sanear el separador reservado	0	no	no	sí	—

Fuente: elaboración propia. Detección por documento: proporción de documentos afectados que el kit en ejecución rechazó. Un mutante sin documentos afectados en la muestra solo puede detectarse con fixtures.